

**基於多模態特徵與堆疊架構之AI邊緣運算釣魚威脅
偵測系統**

**A Phishing Threat Detection System for AI Edge
Computing Based on Multimodal Features and
Stacking Architecture**

文件編號NCHU-MIS-2026-05

國立中興大學資訊管理學系

指導老師：陳育毅教授

專題學生：王昱殊、廖丞帷、陳澤宇

中華民國115年5月14日

目錄

壹、系統計畫書.....	3
一、內容目錄.....	3
二、摘要 (Abstract).....	6
三、系統需求 (System Request).....	7
四、工作計畫(Workplan).....	9
五、需求定義(Requirement Definition).....	13
六、可行性分析(Feasibility Analysis).....	15
七、功能模型 (Functional Model).....	17
八、結構模型 (Structure Model).....	27
九、行為模型 (Behavioral Model).....	38
十、附錄.....	42
貳、系統規格書.....	52
一、系統技術架構 (Technical Architecture).....	53
二、介面規格說明 (Interface Specification).....	54
三、類別與方法 (Class & Method Design).....	55
四、核心模組實作規格 (Core Module Implementation Specification).....	62
五、系統使用流程(System Usage Process).....	78
參、測試計畫(Testing).....	80
一、測試目的.....	81
二、測試環境.....	82
三、單一模型效能測試實驗.....	83
四、堆疊模型效能驗證實驗.....	84
五、實務情境診斷測試.....	90
六、邊緣端效能測試.....	91
七、測試結論.....	91
肆、使用者文件說明(User Document).....	92
一、系統環境要求.....	93
二、系統安裝與配置步驟.....	93
三、使用流程與介面操作.....	94
四、常見問題.....	95
五、故障排除.....	95
伍、結論與未來展望 (Conclusion & Future Work).....	96
一、專案總結.....	97
二、系統限制與不足.....	97
三、未來展望.....	97
陸、參考資料 (References).....	98

壹、系統計畫書

一、內容目錄

- 一、內容目錄
- 二、摘要 (Abstract)
- 三、系統需求 (System Request)
- 四、工作計畫 (Workplan)
- 五、需求定義 (Requirement Definition)
- 六、可行性分析 (Feasibility Analysis)
- 七、功能模型 (Functional Model)
- 八、結構模型 (Structure Model)
- 九、行為模型 (Behavioral Model)
- 十、附錄

二、摘要 (Abstract)

摘要：

網路詐騙手法日益精進，傳統依賴靜態網址規則或黑名單的偵測技術，已難以應對駭客不斷更新的技術與心理戰術。過往研究雖嘗試導入雲端大型語言模型 (LLM) 進行語意分析，卻衍生出嚴重的資料隱私外洩風險與 API 連線延遲問題。為突破此技術瓶頸，提出一套基於本地小型語言模型 與多模態堆疊集成學習的即時偵測系統。

本研究開發「基於多模態特徵與堆疊架構之 AI 邊緣運算釣魚威脅偵測系統」，其目的為解決雲端偵測之隱私疑慮與傳統模型判斷猶豫之痛點。核心貢獻包含：(1) 多模態特徵工程：整合網址統計(45項)、DOM 結構(16項)與本地 Qwen-8B 所提取之語意特徵(15項)，建構 76 維特徵空間，填補傳統方法對於「社交工程意圖識別」之空白；(2) 異質堆疊集成架構：以 XGBoost 為基礎層並透過邏輯回歸 (Logistic Regression) 元模型進行最終決策整合。經實驗證實堆疊後「猶豫區間」大幅下降，且「高自信區間」有所提升；(3) 本地化邊緣推論：利用 CUDA 與 llama-cpp 框架實作可解釋 AI 機制，系統將底層特徵逆向解析為資安邏輯，並依此生成「風險診斷報告」。在確保隱私下提供即時風險評估與診斷報告。本系統在確保資料不出地端的前提下，達成極致的決策穩定度，對零時差釣魚威脅具備高度實務防禦價值。

實驗結果顯示，本系統展現優異的防禦效能，ROC AUC 達 0.9987。更關鍵的是，堆疊架構成功將單一模型的「決策猶豫區間」大幅壓縮至僅剩 0.06%，同時將「高自信區間」推升至 99.69%，展現出極致的決策穩定度。此外，透過 AI 生成之診斷報告增加模型可解釋性，使系統決策邏輯透明化。最終系統封裝為本地 Flask API 與支援 NDJSON 串流技術之 Chrome 擴充功能，實現兼具隱私保護、低延遲與極致偵測效能的落地應用。

關鍵詞：釣魚網站偵測、多模態特徵、堆疊架構、邊緣運算、小型語言模型、語意診斷、可解釋 AI

Abstract:

As cyber-attacks become increasingly sophisticated, traditional detection techniques relying on static URL rules or blacklists are difficult to counter hackers' constantly updating technical and psychological tactics. While previous research attempted to introduce cloud-based Large Language Models (LLMs) for semantic analysis, it led to serious data privacy leakage risks and API connection latency issues. To break through this technical bottleneck, this study proposes a real-time detection system based on a local Small Language Model and multi-modal stacked ensemble learning.

This research developed an "AI Edge Computing Phishing Threat Detection System based on Multi-modal Features and Stacked Architecture" to solve the pain points of privacy concerns in cloud detection and the "decision hesitation" of traditional models. The core contributions include: **(1) Multi-modal Feature Engineering:** Integrating URL statistics (45 items), DOM structure (16 items), and semantic features extracted by a local Qwen-8B (15 items) to construct a 76-dimensional feature space, filling the gap in "Social Engineering Intent Recognition" of traditional methods. **(2) Heterogeneous Stacked Ensemble Architecture:** Utilizing XGBoost as the base layer and Logistic Regression as the meta-model for final decision integration. Experimental results confirm that after stacking, the "hesitation interval" significantly decreases, and the "high-confidence interval" is improved. **(3) Localized Edge Inference:** Utilizing the **CUDA** and **llama-cpp** frameworks to implement an Explainable AI mechanism. The system reverse-parses underlying features into cybersecurity logic and generates "**Risk Diagnosis Reports**" accordingly, providing real-time risk assessments and diagnosis reports while ensuring privacy.

Experimental results show that this system demonstrates superior defensive performance, with an **ROC AUC of 0.9987**. More importantly, the stacking architecture successfully compresses the decision **ambiguity interval** to a mere 0.06% while boosting the **high confidence interval** to 99.69%, exhibiting extreme decision stability. Furthermore, the AI-generated diagnosis reports increase model interpretability, making the system's decision logic transparent. Finally, the system is encapsulated as a local **Flask API** and a **Chrome Extension** supporting **NDJSON** streaming technology, realizing a practical application that features privacy protection, low latency, and ultimate detection performance.

Keywords: Phishing Detection, Multi-modal Features, Stacked Architecture, Edge Computing, Small Language Model, Semantic Diagnosis, Explainable AI

三、系統需求 (System Request)

3.1 開發背景(Project Background)

隨著全球網路服務的普及與數位金融的快速發展，釣魚網站已成為當今最具破壞性的資訊安全威脅之一。攻擊者常透過偽造合法機構（如銀行、社群平台或電子郵件服務）的登入頁面，結合社交工程手法，建構出極具欺騙性的偽造網頁，誘騙使用者輸入帳號密碼、信用卡號等敏感資訊。傳統的釣魚網站偵測技術多半依賴黑名單或基於網址字元特徵的靜態規則比對。

3.2 問題發現與現況分析 (Problem Statement)

經由對現有偵測技術的研究與市場現狀觀察，本研究發現以下核心問題：

1. 偵測技術之局限性：傳統偵測方案多仰賴黑名單或靜態規則比對。面對具備高度偽裝性、甚至是「零時差」的釣魚網站，單一模型或規則難以捕捉其多變的特徵。
2. 語意理解能力的缺失：現有偵測器多著重於統計特徵（如 URL 長度），缺乏對網頁文本情感（如營造急迫感、脅迫感）的深度理解，這正是社交工程攻擊最核心的環節。
3. 隱私洩露之隱憂：整合大型語言模型雖能提升偵測率，但目前大多數方案依賴雲端 API（如 OpenAI），導致使用者瀏覽的私密數據必須上傳雲端，產生嚴重的資安與隱私洩漏風險。
4. 對大規模資料庫的過度依賴：許多高效能偵測技術須仰賴後端巨量資料庫之比對與檢索。這導致系統在面對新生成的釣魚網頁時，會因資料庫尚未收錄而產生偵測真空期，且頻繁的遠端查詢亦大幅降低了防護的即時性。
5. 運算成本與延遲：頻繁調用雲端 LLM API 會產生昂貴的運算費用，且受限於網路延遲，難以在使用者瀏覽網頁的當下提供即時偵測回饋。

3.3 業務需求 (Business Needs)

為解決上述問題，本系統旨在達成以下目標：

- 多模態融合：整合網址、結構與語意三個維度的特徵，提升偵測廣度。
- 本地化推論：將 AI 運算遷移至邊緣端，實現數據不離地，確保絕對隱私與即時響應。
- 去中心化分析：不依賴即時資料庫查詢，轉而強化對網頁特徵的自主辨識能力，實現「瀏覽即偵測」的目標。
- 高精準決策：利用堆疊架構整合多個異質模型，降低誤判率與漏報率。

3.4 業務價值 (Business Value)

- 有形價值：降低因詐騙導致的個資外洩與金錢損失，提升企業品牌保護力。
- 無形價值：建立使用者對網路環境的信任感，並展現本地化 AI (Edge AI) 在資安領域的創新應用範例。

3.5 預期效益 (Expected Benefits)

本系統實作後，預期達成以下具體效益：

1. 提升零時差攻擊偵測率：透過76維多模態特徵與堆疊架構，系統能識別尚未被收錄於黑名單中的新型釣魚頁面，強化動態防禦能力。
2. 實現絕對隱私之語意分析：利用本地邊緣運算技術，使LLM分析過程完全在用戶端硬體執行，徹底杜絕瀏覽內容外流至雲端服務商的風險。
3. 優化資安預警之互動體驗：透過非同步串流技術，系統能即時回報掃描進度並生成自然語言診斷報告，讓使用者「知其然更知其所以然」。
4. 降低企業與個人之防護成本：去中心化的推論架構無需負擔昂貴的雲端 API 調用費用，且能在斷網或低頻寬環境下維持核心偵測功能。

3.6 特殊限制與考量 (Special Issues)

- 系統須運行於具備NVIDIA GPU加速之設備，以確保LLM推論效能。
- 偵測流程須與現有瀏覽器插件機制無縫整合，不干擾使用者正常瀏覽體驗。

四、工作計畫(Workplan)

4.1 開發方法 (Development Methodology)

本專案採用「雛型法」與「迭代開發」模式。系統核心採用 Stacking 整合學習架構：

- Level 0 (基礎層)：分別針對URL(XGBoost)、HTML(XGBoost)及文本語意(XGBoost) 進行初階預測。
- Level 1 (元模型層)：利用Logistic Regression整合基礎層之預測機率，輸出最終風險指數。

4.2 技術架構 (Technical Stack)

為實現高效的邊緣運算與複雜模型推理，系統選用以下技術：

- 後端架構：Python 3.10/Flask Web Server。
- AI 運算核心：XGBoost, TensorFlow/Keras, Joblib:llama-cpp-python (Qwen-8B)。
- 前端介面：JavaScript(Chrome Extension)，利用chrome.scripting API進行DOM擷取。
- 硬體加速：CUDA Toolkit/NVIDIA GPU Tensor Cores。

4.3 系統工作分解結構 (WBS)

本系統開發任務拆解如圖4.3.1。

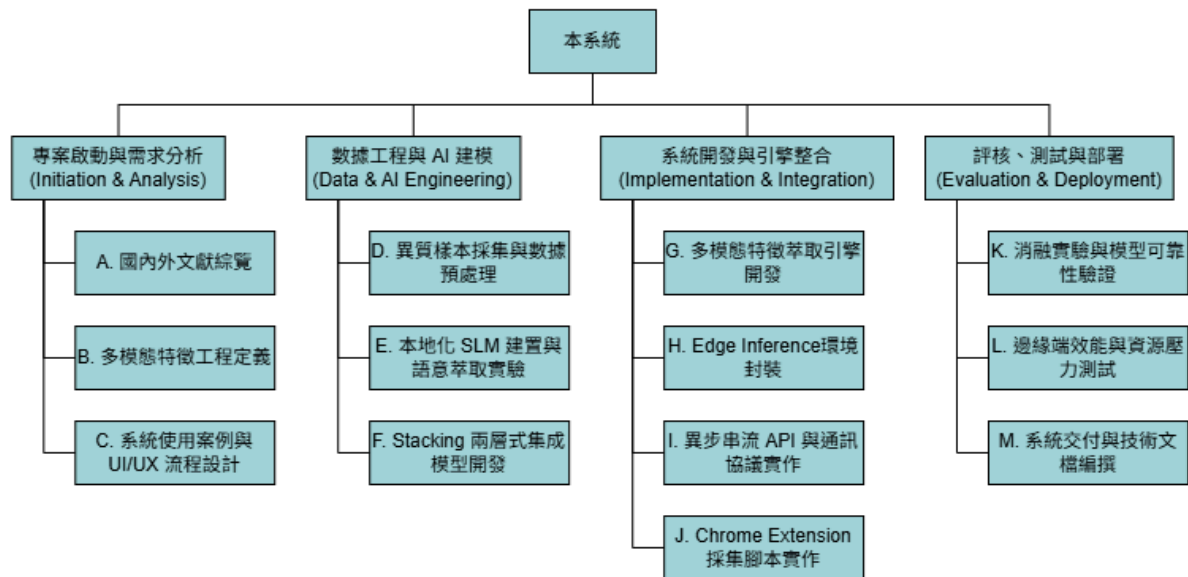


圖4.3.1：本系統之工作分類架構圖

4.4 工作項目定義與負責人 (Work Item Definition & Responsibility)

代號	項目名稱	定義與說明	預期產出	負責人
A	國內外文獻綜覽	研讀相關論文，分析他人研究方法與發現	文獻探討報告	王昱殊、廖丞帷、陳澤宇
B	多模態特徵工程定義	確立 76 維多模態特徵 (URL、HTML、文本語意) 之計算方法。	特徵定義表	王昱殊
C	系統使用案例與 UI/UX 流程設計	規劃擴充套件介面佈局、偵測互動流程及風險提示視覺設計。	介面原型	王昱殊
D	異質樣本採集與數據預處理	執行樣本採集、標記、缺失值處理、標籤化及標準化。	可用之訓練資料集	王昱殊
E	本地化 SLM 建置與語意萃取實驗	部署本地Qwen-8B模型，實作 Prompt Tuning並生成文本語意特徵。	語意特徵向量集	王昱殊
F	Stacking 兩層式集成模型開發	實作第0層異質基礎模型訓練與第一層元模型決策融合邏輯。	訓練完成之權重檔	王昱殊
G	多模態特徵萃取引擎開發	使用Python撰寫特徵提取類別，確保三路特徵數據對齊。	特徵提取模組	王昱殊
H	Edge Inference 環境封裝	配置CUDA工具鏈與llama-cpp核心，優化邊緣端推論效能。	本地推論運行環境	王昱殊
I	異步串流 API 與通訊協議實作	建立Flask Server，實作支援 NDJSON 格式之非同步串流API。	API 規格與伺服器程式	王昱殊
J	Chrome 擴充套件採集腳本實作	撰寫擴充套件之內容腳本，實現動態 DOM 與文本數據擷取。	套件採集程式碼	王昱殊
K	消融實驗與模型可靠性驗證	執行各模態模型實驗與集成模型之對比實驗。	實驗數據	王昱殊、廖丞帷
L	邊緣端效能與資源壓力測試	測試邊緣端 GPU 資源佔用率、記憶體消耗與極端網頁下之延遲。	效能分析報告	王昱殊、廖丞帷
M	系統交付與技術文檔編撰	彙整開發過程，編撰系統操作手冊、畢業論文與成果演示。	畢業論文、開發手冊	王昱殊、廖丞帷、陳澤宇

4.5 工作項目流程網狀圖 (Work Item Network Diagram / PERT Chart)

本系統工作項目流程可拆解如圖4.5.1之網狀圖

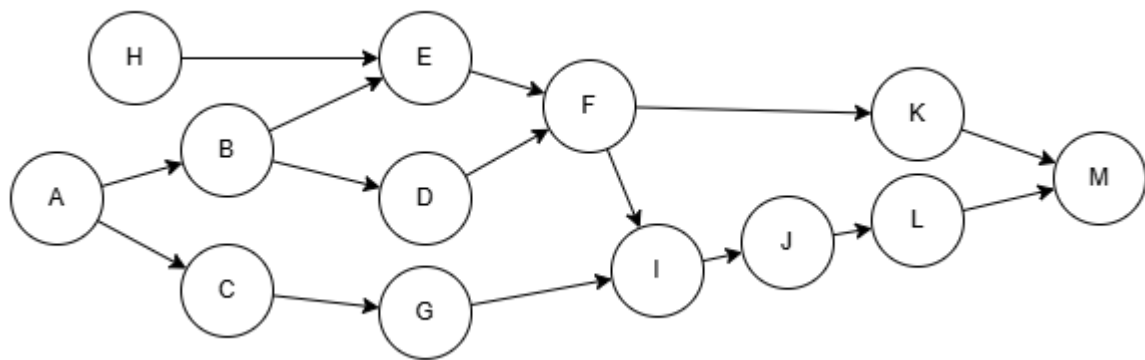


圖4.5.1：本系統之網狀圖

4.6 專案進度甘特圖 (Project Gantt Chart)

本系統開發時程表可拆解如圖4.6.1

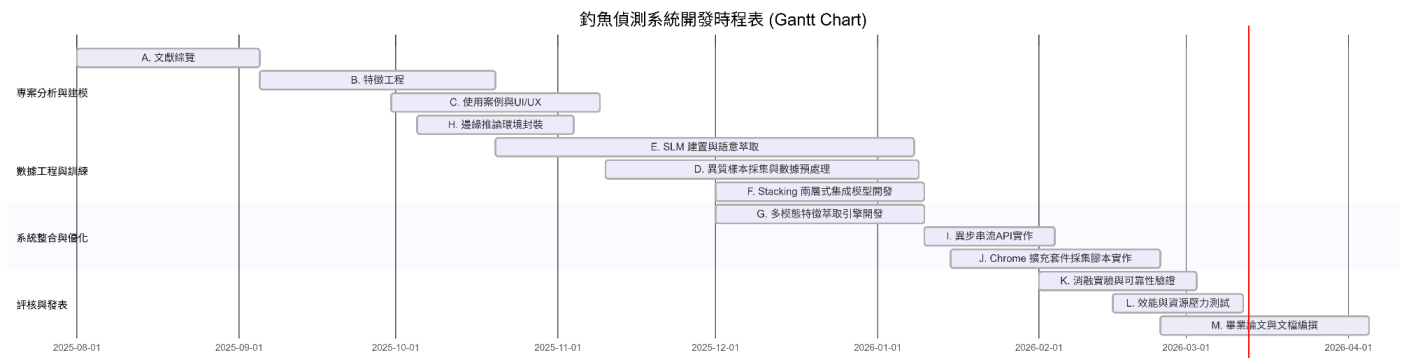


圖4.6.1：甘特圖

五、需求定義(Requirement Definition)

5.1 模型研究與建立階段需求 (Model-Centric Requirements)

定義模型開發過程中的核心功能，包含多維特徵之運算品質與堆疊架構之建置。

需求項目	需求描述
文本自動標記	透過本地 SLM (Qwen3-8B) 實現資料集之自動語意標記，將非結構化文本轉為 15 項語意特徵。
多維特徵定義	訂定特徵定義與萃取邏輯
本地語言模型建置	開發本地SLM環境，並導入語言模型(Qwen3-8B-Q4_K_S)
異質特徵對齊機制	確保網址、結構與文本特徵在模型輸入前完成 標準化、標籤化、型態轉換及相關預處理與對齊。
堆疊層級預測實作	系統須實作 Level 0與 Level 1兩層式預測邏輯。
消融實驗數據產出	記錄並比較單一模型與堆疊模型之效能指標表現。

5.2 系統實作與整合階段需求 (System-Centric Requirements)

定義系統在使用者終端運作時的功能表現與互動流程。

需求項目	需求描述
端點數據擷取功能	Chrome 插件須能動態注入腳本，即時擷取 DOM、URL 與可見文本資料。
NDJSON串流回饋	系統須實作非同步進度回饋機制，於掃描過程中即時更新前端進度文字。
AI 診斷報告生成	系統須整合相關數據特徵與模型判定結果，並藉由SLM生成包含「風險指數」與「判斷理由」之中文報告。
異質特徵對齊機制	確保網址、結構與文本特徵在模型輸入前完成 標準化、標籤化、型態轉換及相關預處理與對齊。
容錯與防護機制	對於非預期之狀況發生，系統須能有正確對應方式，而非程式崩潰。

5.3 非功能性需求 (Non-functional Requirements)

類別	需求描述	品質指標
隱私安全性	全本地化數據處理	所有特徵提取與模型推論流程嚴禁將網頁內容數據傳輸至外部雲端伺服器。
時間效能	邊緣推論時延	自使用者點擊掃描至看到診斷報告之總時長，於理想環境下應低於25秒。
穩定性	硬體資源監控	系統推論期間應透過量化技術，確保佔用不影響作業系統之基礎運行，並避免溢出錯誤。
部屬相容性	瀏覽器Manifest V3	前端擴充功能須遵守 Google Chrome 之最新擴充功能安全性規範。

5.4 系統開發環境與資源需求

1. 模型開發環境

- 硬體:NVIDIA RTX 30系列(以上)用於訓練與量化測試。
- 軟體:Scikit-learn、Keras、XGBoost Library、Qwen GGUF Model。

2. 系統部屬環境

- 硬體：支持CUDA加速之個人電腦與前述之顯卡。
- 語言：Python、JavaScript。
- 框架：Flask Server、llama-cpp-python。

六、可行性分析(Feasibility Analysis)

6.1 技術可行性 (Technical Feasibility)

1. **邊緣運算與量化技術**：目前 SLM (Qwen3-8B) 已可透過GGUF格式與4-bit量化技術大幅降低VRAM需求。配合llama-cpp-python與CUDA硬體加速，已實測能於RTX 3060(以上)之裝置運行。
2. **多模態特徵萃取之成熟度**：URL 與 HTML 結構特徵分析技術已發展成熟，結合BeautifulSoup4與正則表達式可實現高效能靜態解析。
3. **整合學習 (Stacking) 架構**：Scikit-learn與TensorFlow框架對於多模型的整合具備完整支援，Meta-model之決策融合邏輯與技術路徑清晰。
4. **瀏覽器擴充功能規範**：Chrome Extension Manifest V3提供穩定的scripting與非同步通訊介面，能確保前端採集與後端之銜接。

6.2 經濟可行性 (Economic Feasibility)

1. **營運成本顯著降低**
 - 傳統方案：若使用雲端 LLM API (如 GPT-4)，每筆偵測約需支付 0.03~0.1 美元，當用戶量成長時，成本將呈線性爆炸。
 - 本專案方案：採用本地化邊緣運算，除了設備購置成本外，API調用費用為零。這使得系統在長期運行下具備極高的成本競爭力。
2. **無形資產價值**：本系統可有效降低因釣魚詐騙導致的個資外洩賠償、品牌商譽損失及金融犯罪成本。對於重視隱私的企業而言，數據不離地的特性具備極高的商用轉化潛力。

6.3 作業可行性 (Operational Feasibility)

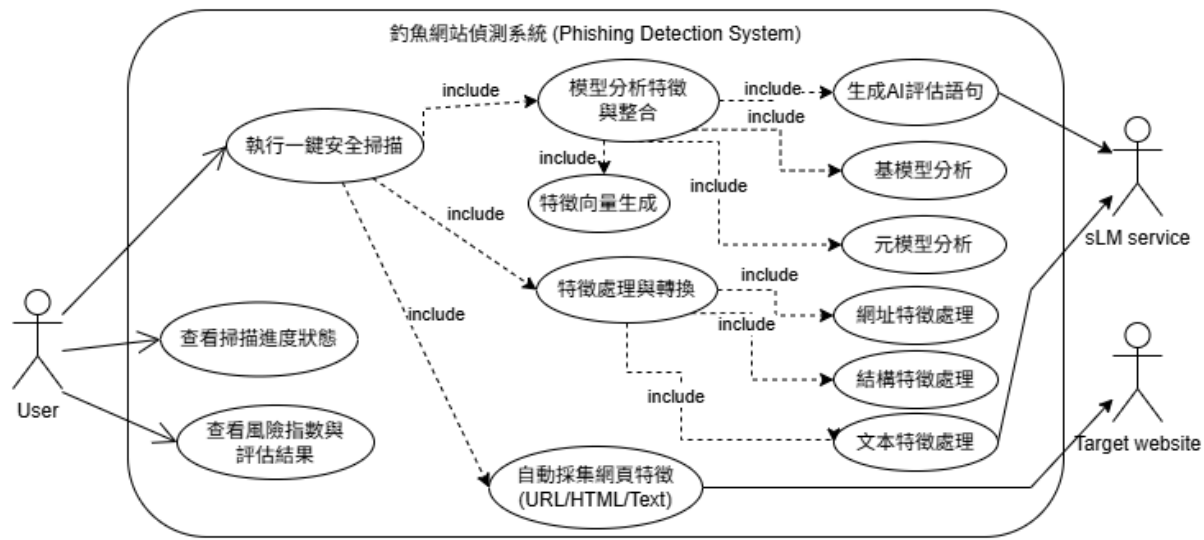
1. **無干擾式使用者體驗**：透過瀏覽器外掛形式呈現，使用者無需改變瀏覽習慣，只需一鍵點擊即可獲得分析報告。
2. **自動化診斷報告**：SLM生成的語意分析能以自然語言解釋「為何這是釣魚網站」，解決了傳統工具「只給結果、不給理由」的黑盒子問題，增加使用者對系統的信任度。
3. **隱私保障機制**：由於資料完全在本地處理，免除了使用者對於「掃描網頁會導致私密數據流向第三方」的心理壓力，提升了系統的接受度。

6.4 法律與環境可行性 (Legal & Environmental Feasibility)

1. **符合法規趨勢：**在 GDPR 與《個人資料保護法》日益嚴格的趨勢下，本系統的「數據不離地」特徵完美契合隱私保護之法律規範。
2. **環境適應力：**系統不依賴大規模雲端伺服器集群之即時通訊，在低頻寬或不穩定之網路環境下仍能藉由邊緣運算特性維持偵測功能，展現強大的運算韌性。

七、功能模型 (Functional Model)

7.1 使用案例圖 (Use Case Diagram)



7.2 使用案例說明 (Use Case Descriptions)

編號： USC-001	使用案例名稱： 執行一鍵安全掃描	行為者： User
前置條件： 使用者已啟動瀏覽器並安裝本系統之 Chrome Extension。		觸發事件： 使用者點擊「執行一鍵安全掃描」按鈕。
正常程序： 1. 前端系統執行「自動採集網頁特徵(URL、HTML、Text)」。 2. 前端系統傳送網址、網頁結構、網頁文本至後端。 3. 後端系統執行「特徵處理與轉換」。 4. 後端系統執行「模型分析特徵與整合」。 5. 後端系統回傳風險指數與AI評估至前端給使用者。		
後置條件： 系統成功產出風險指數與AI評估報告。		例外狀況： 執行過程錯誤將返回提示訊息。

編號: USC-002	使用案例名稱: 自動採集網頁特徵(URL、HTML、Text)	行為者: Target website
前置條件: 已觸發「執行一鍵安全掃描」流程。		觸發事件: 使用者點擊「執行一鍵安全掃描」按鈕
正常程序: 1. 前端系統擷取Target website之網址、網頁結構、網頁文本。		
後置條件: 前端系統成功擷取Target website之網址、網頁結構、網頁文本。		例外狀況: 執行過程錯誤將返回提示訊息。

編號: USC-003	使用案例名稱: 特徵處理與轉換	行為者: 系統
前置條件: 已觸發「執行一鍵安全掃描」流程。		觸發事件: 前端系統傳送網址、網頁結構、網頁文本至後端。
正常程序: 1. 後端系統執行「網址特徵處理」，取得網址特徵集。 2. 後端系統執行「結構特徵處理」，取得結構特徵集。 3. 後端系統執行「文本特徵處理」，取得文本特徵集。 4. 後端系統載入標準化標準。 5. 將特徵集特定欄位根據標準化標準標準化。 6. 將特徵集特定欄位根據格式標籤化。		
後置條件: 成功預處理三個特徵集		例外狀況: 執行過程錯誤將返回提示訊息

編號: USC-004	使用案例名稱: 網址特徵處理	行為者: 系統
前置條件: 已觸發「執行一鍵安全掃描」流程。		觸發事件: 後端系統接受到預處理之網址。
正常程序: 1. 根據預定義之45項統計邏輯計算萃取網址特徵。		
後置條件: 成功萃取出網址特徵集		例外狀況: 執行過程錯誤將返回提示訊息

編號: USC-005	使用案例名稱: 結構特徵處理	行為者: 系統
前置條件: 已觸發「執行一鍵安全掃描」流程。		觸發事件: 後端系統接受到預處理之網頁結構。
正常程序: 1. 根據預定義之16項統計邏輯計算萃取結構特徵。		
後置條件: 成功萃取出結構特徵集		例外狀況: 執行過程錯誤將返回提示訊息

編號: USC-006	使用案例名稱: 文本特徵處理	行為者: 系統
前置條件: 已觸發「執行一鍵安全掃描」流程。		觸發事件: 後端系統接受到預處理之網頁文本。
正常程序: 1. 結合提示詞與網頁文本，傳入語言模型。 2. 語言模型根據指定格式回傳包含預定義的15項文本特徵的回覆。 3. 將回覆整理取得網頁文本特徵。		
後置條件: 成功萃取出文本特徵集		例外狀況: 執行過程錯誤將返回提示訊息

編號: USC-007	使用案例名稱: 模型分析特徵與整合	行為者: 系統
前置條件: 已觸發「執行一鍵安全掃描」流程。		觸發事件: 後端系統取得三個已經預處理的特徵集
正常程序: 1. 確認各特徵集之特徵是否有誤。 2. 後端系統執行「基模型分析」，對結果執行「特徵向量生成」。 3. 後端系統執行「元模型分析」，取得最終結果，並加工成風險指數。 4. 根據最終結果與特徵集，製作理由清單。 5. 後端系統執行「生成AI評估語句」，取得AI評估。		
後置條件: 成功完成預測與取得AI評估		例外狀況: 執行過程錯誤將返回提示訊息

編號: USC-008	使用案例名稱: 基模型分析	行為者: 三個基模型
前置條件: 已觸發「執行一鍵安全掃描」流程。		觸發事件: 特徵集特徵檢查無誤
正常程序: 1. 將網址特徵輸入網址模型，回傳預測結果。 2. 將結構特徵輸入結構模型，回傳預測結果。 3. 將文本特徵輸入文本模型，回傳預測結果。		
後置條件: 成功取得三個基模型的預測結果		例外狀況: 執行過程錯誤將返回提示訊息

編號: USC-009	使用案例名稱: 特徵向量生成	行為者: 系統
前置條件: 已觸發「執行一鍵安全掃描」流程。		觸發事件: 取得三個基模型的預測結果
正常程序: 1. 將三個基模型結果合併，作為新特徵向量。		
後置條件: 成功產生特徵向量		例外狀況: 執行過程錯誤將返回提示訊息

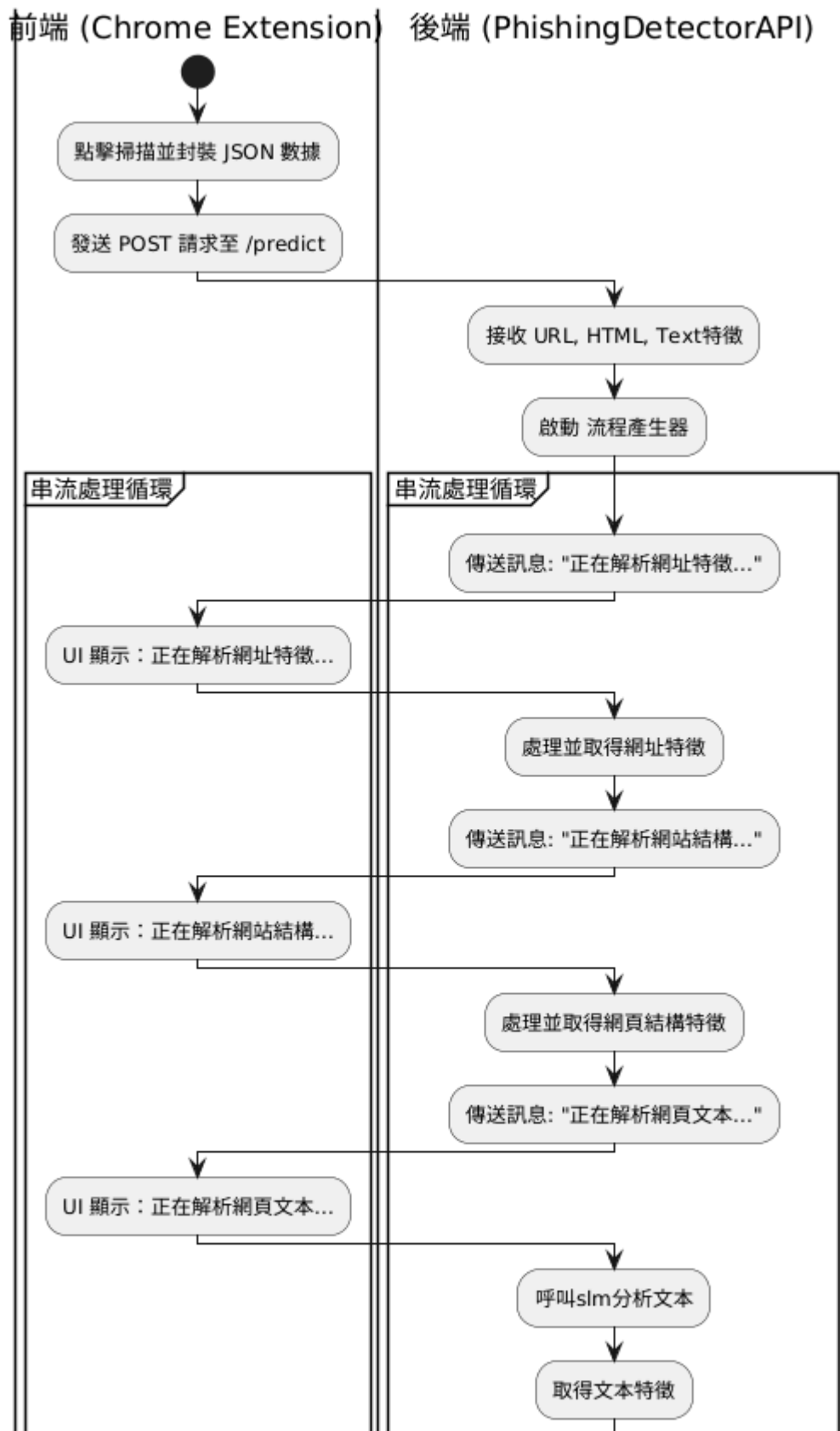
編號: USC-010	使用案例名稱: 元模型分析	行為者: 元模型
前置條件: 已觸發「執行一鍵安全掃描」流程。		觸發事件: 產生特徵向量
正常程序: 1. 將特徵向量輸入至元模型(level 0)，產生最終機率結果。		
後置條件: 成功產生最終機率結果		例外狀況: 執行過程錯誤將返回提示訊息

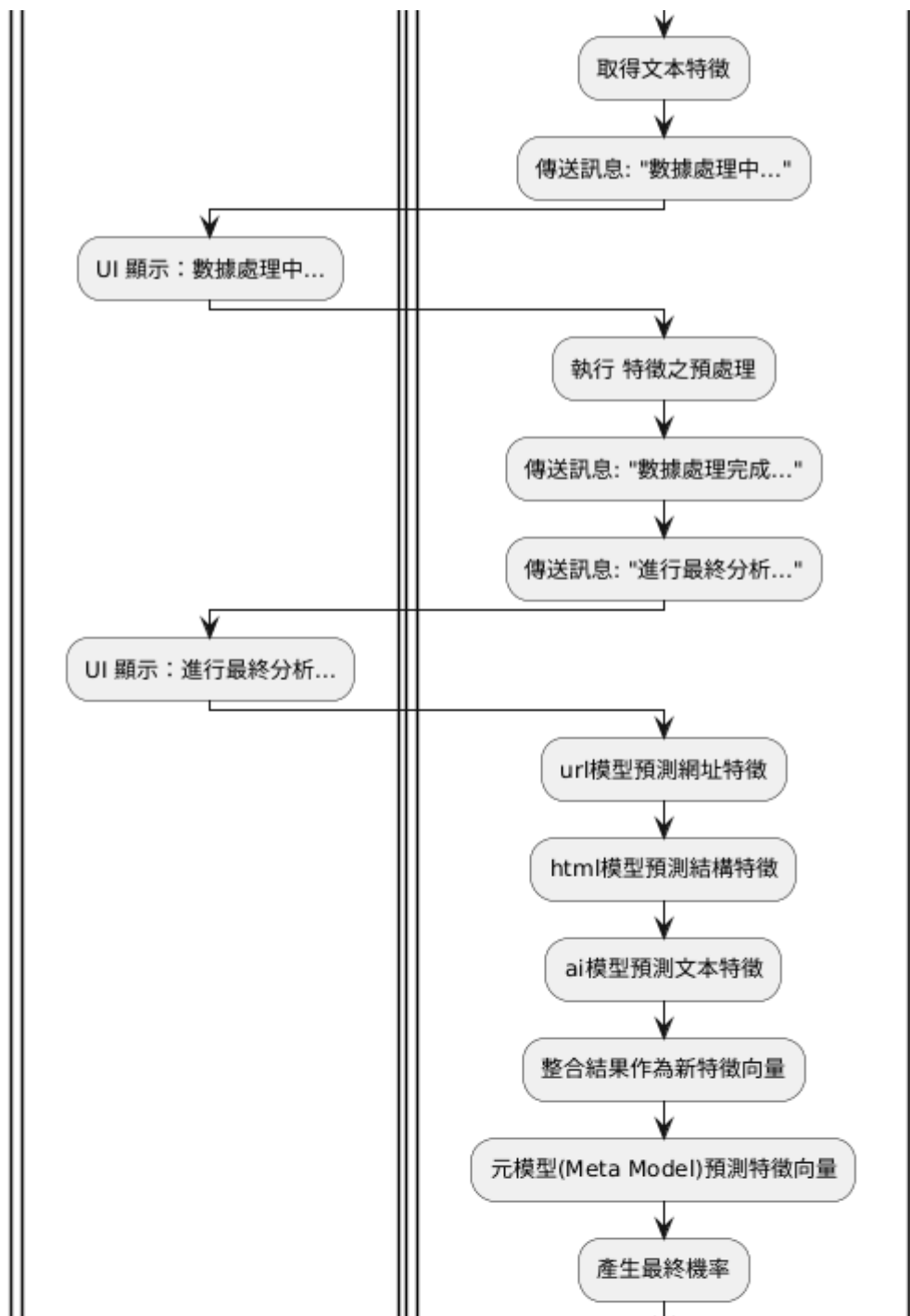
編號: USC-011	使用案例名稱: 生成AI評估語句	行為者: 本地語言模型
前置條件: 已觸發「執行一鍵安全掃描」流程。		觸發事件: 完成理由清單與堆疊模型預測
正常程序: 1. 將理由清單、模型預測結果，結合提示詞輸入至本地語言模型。 2. 取得語言模型生成之評估語句。		
後置條件: 成功產生AI評估語句		例外狀況: 執行過程錯誤將返回提示訊息

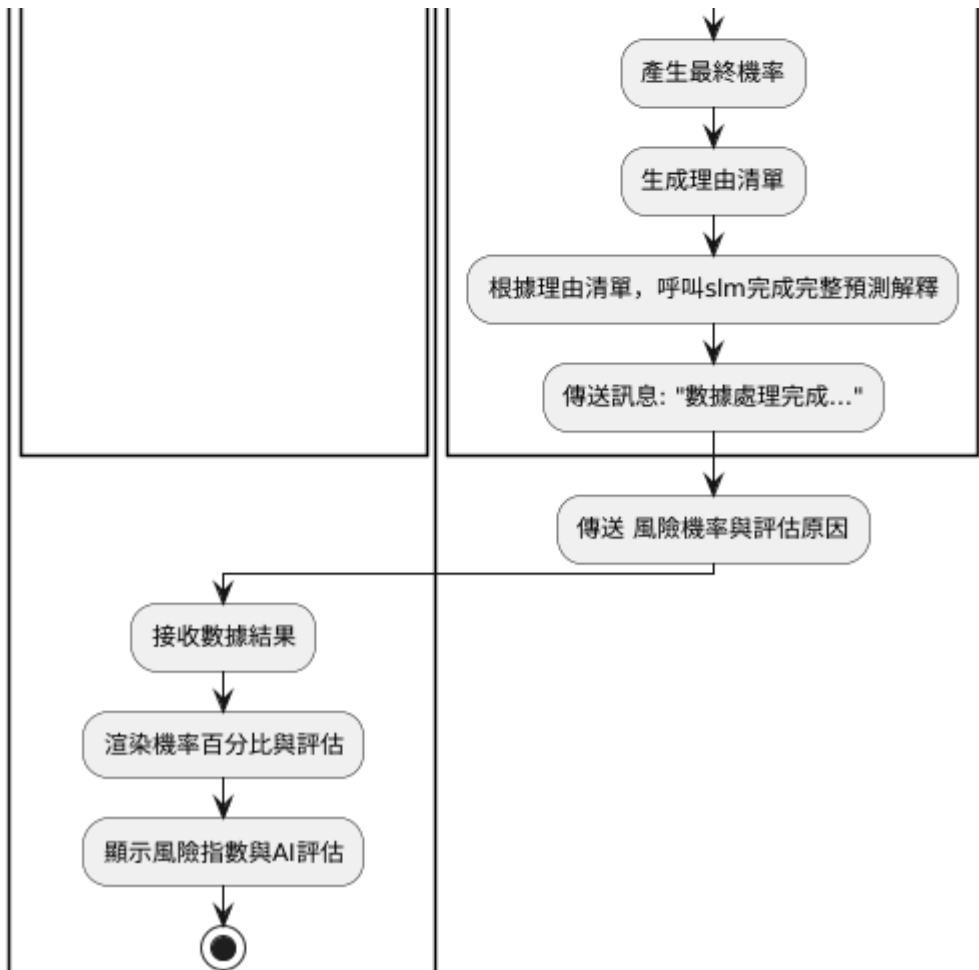
編號: USC-012	使用案例名稱: 查看掃描進度狀態	行為者: 使用者、系統
前置條件: 已觸發「執行一鍵安全掃描」流程。		觸發事件: 接收到來自後端系統的進度回報
正常程序: 1. 接收並顯示「正在解析網址特徵...」。 2. 接收並顯示「正在解析網站結構...」。 3. 接收並顯示「正在解析網頁文本...」。 4. 接收並顯示「數據處理中...」。 5. 接收並顯示「數據處理完成!」。 6. 接收並顯示「進行最終分析...」。 7. 接收並顯示「最終分析完成!」。		
後置條件: 成功產生AI評估語句		例外狀況: 執行過程錯誤將返回提示訊息

編號: USC-013	使用案例名稱: 查看風險指數與評估結果	行為者: 使用者
前置條件: 已觸發「執行一鍵安全掃描」流程。		觸發事件: 接收到來自後端系統的掃描結果
正常程序: 1. 前端系統將後端回傳之風險指數及AI評估語句依據格式顯示。		
後置條件: 前端呈現出後端掃描結果		例外狀況: 執行過程錯誤將返回提示訊息

7.3 活動流程圖 (Activity Diagram)

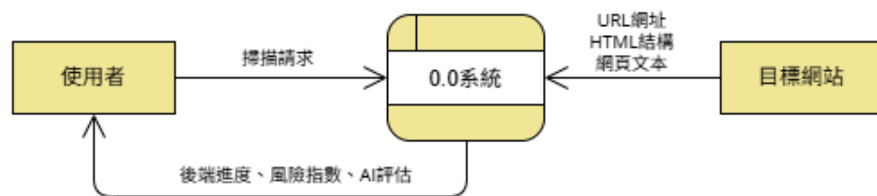




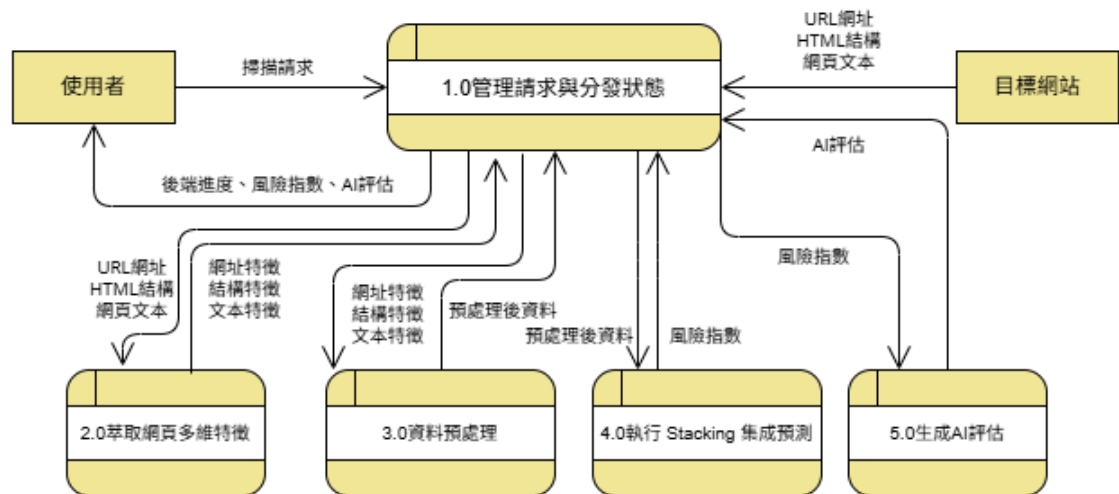


7.4 資料流程圖 (Data Flow Diagram)

1. 內容層圖(DFD 0)

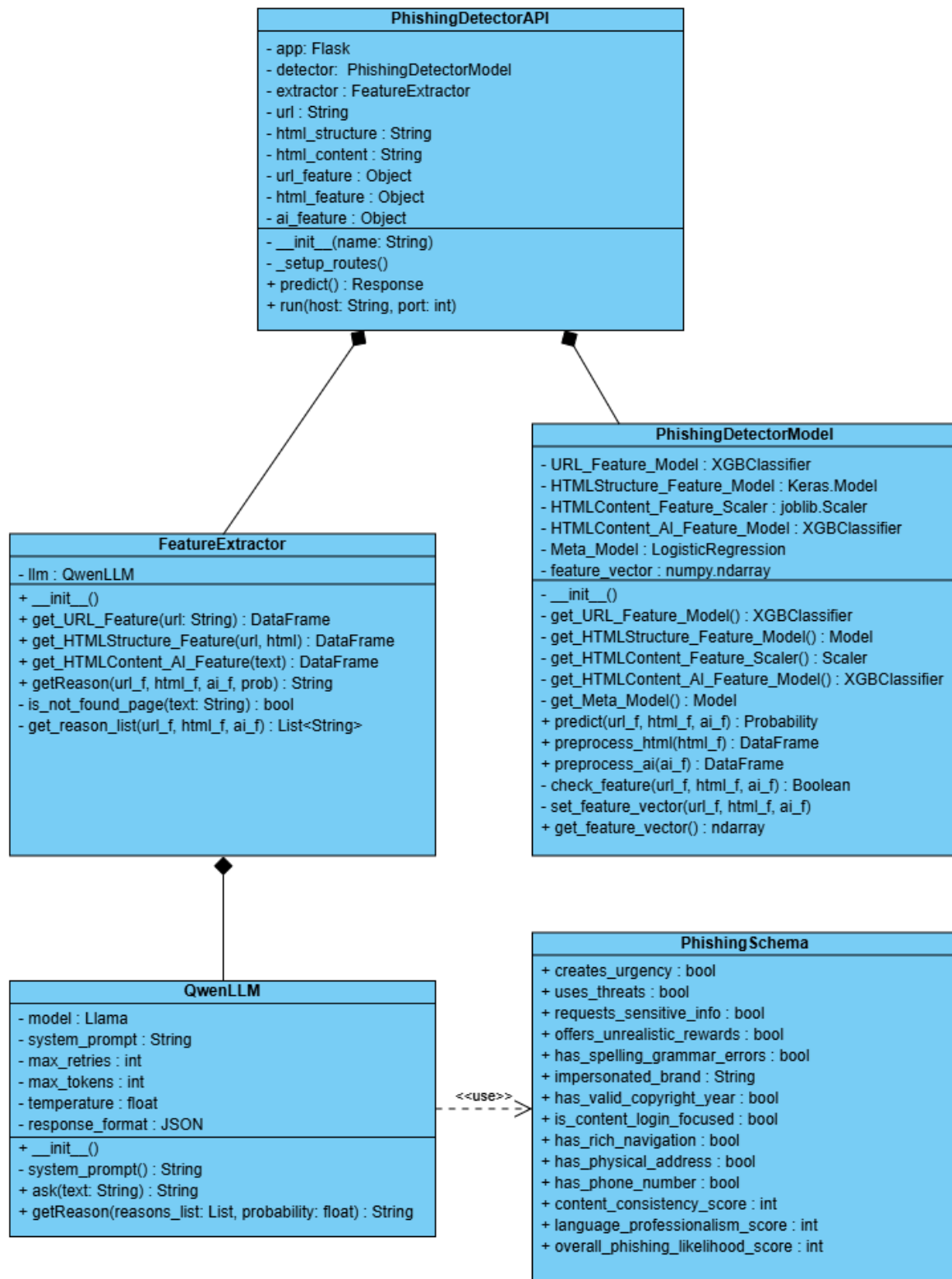


2. 資料流層級圖(DFD 1)



八、結構模型 (Structure Model)

8.1 系統類別圖 (Class Diagram)



8.2 資料字典 (Data Dictionary)

定義每個特徵與計算方式之特徵工程定義

1. URL 維度特徵 (URL-based Features) - 共 45 項

特徵名稱	資料型態	說明
length_url	Integer	URL的字元長度。
length_hostname	Integer	主機名稱 (Hostname) 的字元長度。
ip	Boolean	檢查 Hostname 是否為 IPv4/v6 地址。
nb_dots	Integer	點號數量。
nb_hyphens	Integer	連字號數量。
nb_at	Integer	@ 符號數量。
nb_qm	Integer	問號數量。
nb_and	Integer	& 符號數量。
nb_or	Integer	符號數量。

nb_eq	Integer	= 符號數量。
nb_underscore	Integer	_ 符號數量。
nb_tilde	Integer	~ 符號數量。
nb_percent	Integer	% 符號數量。
nb_slash	Integer	/ 符號數量。
nb_star	Integer	* 符號數量。
nb_colon	Integer	: 符號數量。
nb_comma	Integer	, 符號數量。
nb_semicolumn	Integer	; 符號數量。
nb_dollar	Integer	\$ 符號數量。
nb_space	Integer	空格數量。
nb_www	Boolean	檢查是否有 "www"。

nb_com	Boolean	檢查是否有 ".com" 。
nb_dslash	Boolean	檢查是否有 "/" 且不在協議部分。
http_in_path	Boolean	檢查路徑中是否有 "http" 或 "https"
https_token	Boolean	檢查是否為 HTTPS。
ratio_digits_url	Float	數字佔整個 URL 字串的比例。
ratio_digits_host	Float	數字佔整個 hostname 字串的比例。
punycode	Boolean	檢查是否為 Punycode 編碼
port	Boolean	檢查是否有指定 Port
tld_in_path	Boolean	頂級域名是否出現在路徑中。
tld_in_subdomain	Boolean	頂級域名是否出現在子域名中。
nb_subdomains	Integer	計算子域名數量。
abnormal_subdomain	Boolean	子域名是否異常。

prefix_suffix	Boolean	檢查域名是否有前綴或後綴符號，如 '-'。
path_extension	Boolean	檢查路徑中是否有檔案副檔名。
length_words_raw	Integer	URL 中所有單詞的總長度。
char_repeat	Boolean	是否有連續重複 2 次以上的字元。
shortest_word_host	Integer	主機名稱中最短單詞的長度。
shortest_word_path	Integer	路徑中最短單詞的長度。
longest_words_raw	Integer	URL 中最長單詞的長度。
longest_word_host	Integer	主機名稱中最長單詞的長度。
longest_word_path	Integer	路徑中最長單詞的長度。
avg_words_raw	Float	URL 中單詞的平均長度。
avg_word_host	Float	主機名稱中單詞的平均長度。
avg_word_path	Float	路徑中單詞的平均長度。

2. HTML 結構維度特徵 (HTML-based Features) - 共 16 項

特徵名稱	資料型態	說明
phish_hints	Boolean	內容中是否存在常見的釣魚提示詞語。
domain_in_brand	Boolean	網站內容提及的品牌名稱是否與域名一致。
nb_hyperlinks	Integer	網頁中超連結的總數。
ratio_inHyperlinks	Float	內部超連結的比例。
ratio_extHyperlinks	Float	外部超連結的比例。
ratio_extRedirection	Float	外部重新導向的比例。
ratio_extErrors	Float	外部連結中返回錯誤的比例。
external_favicon	Boolean	網站是否使用來自外部域名的 Favicon。

links_in_tags	Integer	特定 HTML 標籤（如 <a>、<script>）中連結的數量。
ratio_extMedia	Float	外部媒體（圖片、音頻、視頻）的比例
safe_anchor	Boolean	錨點連結是否安全（如避免指向可疑外部網站）。
empty_title	Boolean	網頁標題是否為空。
domain_in_title	Boolean	域名是否出現在網頁標題中。
domain_with_copyright	Boolean	網站的版權資訊中是否包含域名。
has_meta_refresh	Boolean	偵測 meta 轉向。
has_js_redirect	Boolean	偵測 JavaScript 轉向。

3. AI 語意維度特徵 (AI Semantic Features) - 共 15 項

特徵名稱	資料型態	說明
creates_urgency	Boolean	檢查文本是否營造急迫感（如：限時處理、立即行動）。
uses_threats	Boolean	檢查是否包含威脅語句（如：帳號將被停用、面臨法律責任）。
requests_sensitive_info	Boolean	檢查是否要求敏感資安資訊（如：密碼、信用卡號、身分證字號）。
offers_unrealistic_rewards	Boolean	檢查是否提供不切實際的獎勵（如：中大獎、高額贈禮）。
has_spelling_grammar_errors	Boolean	檢查文本是否存在明顯拼字或語法錯誤。
impersonated_brand	String	識別文本試圖冒充的特定品牌名稱。
has_valid_copyright_year	Boolean	檢查頁尾是否有標註正確且年份合理的版權宣告。
is_content_login_focused	Boolean	檢查內容是否過度集中於登入、驗證等功能，缺乏一般公司資訊。

has_rich_navigation	Boolean	檢查是否有豐富的導覽連結（如：關於我們、徵才、投資者關係）。
has_physical_addresses	Boolean	檢查是否明確標示實體辦公地點地址。
has_phone_number	Boolean	檢查是否提供有效的客服聯絡電話。
content_consistency_score	Integer	(0-10) 評估頁面標題、內文與頁尾資訊的語義一致性。
language_professionalism_score	Integer	(0-10) 評估文本語言的專業度與官網語氣契合度。
overall_phishing_likelihoood_score	Integer	(0-10) 綜合上述所有文本特徵給出的最終釣魚風險評分。

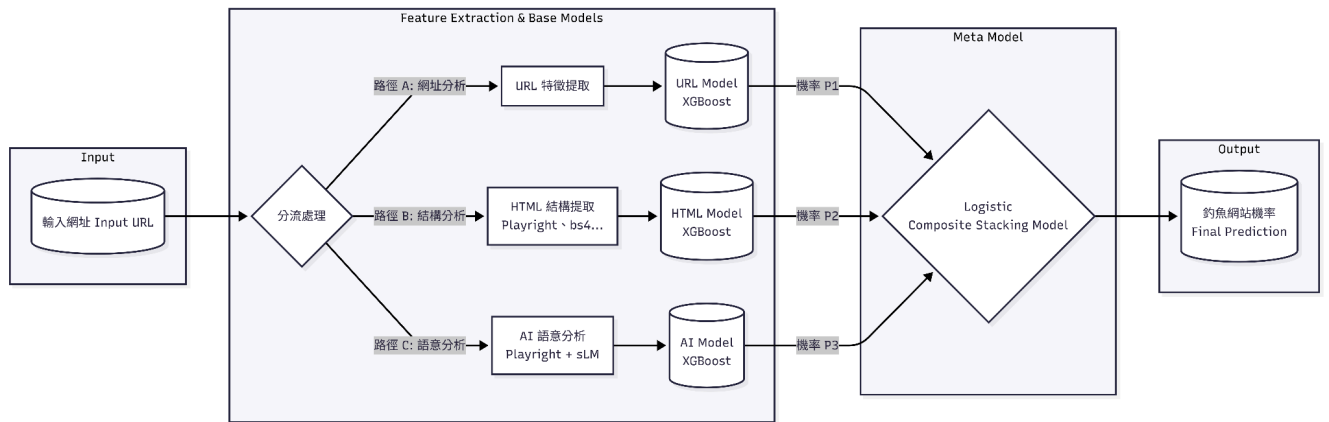
8.3 模型架構 (Model Architecture)

1. SLM模型

```
class PhishingSchema(BaseModel):
    creates_urgency: bool
    uses_threats: bool
    requests_sensitive_info: bool
    offers_unrealistic_rewards: bool
    has_spelling_grammar_errors: bool
    impersonated_brand: str
    has_valid_copyright_year: bool
    is_content_login_focused: bool
    has_rich_navigation: bool
    has_physical_address: bool
    has_phone_number: bool
    content_consistency_score: int = Field(ge=0, le=10)
    language_professionalism_score: int = Field(ge=0, le=10)
    overall_phishing_likelihood_score: int = Field(ge=0, le=10)

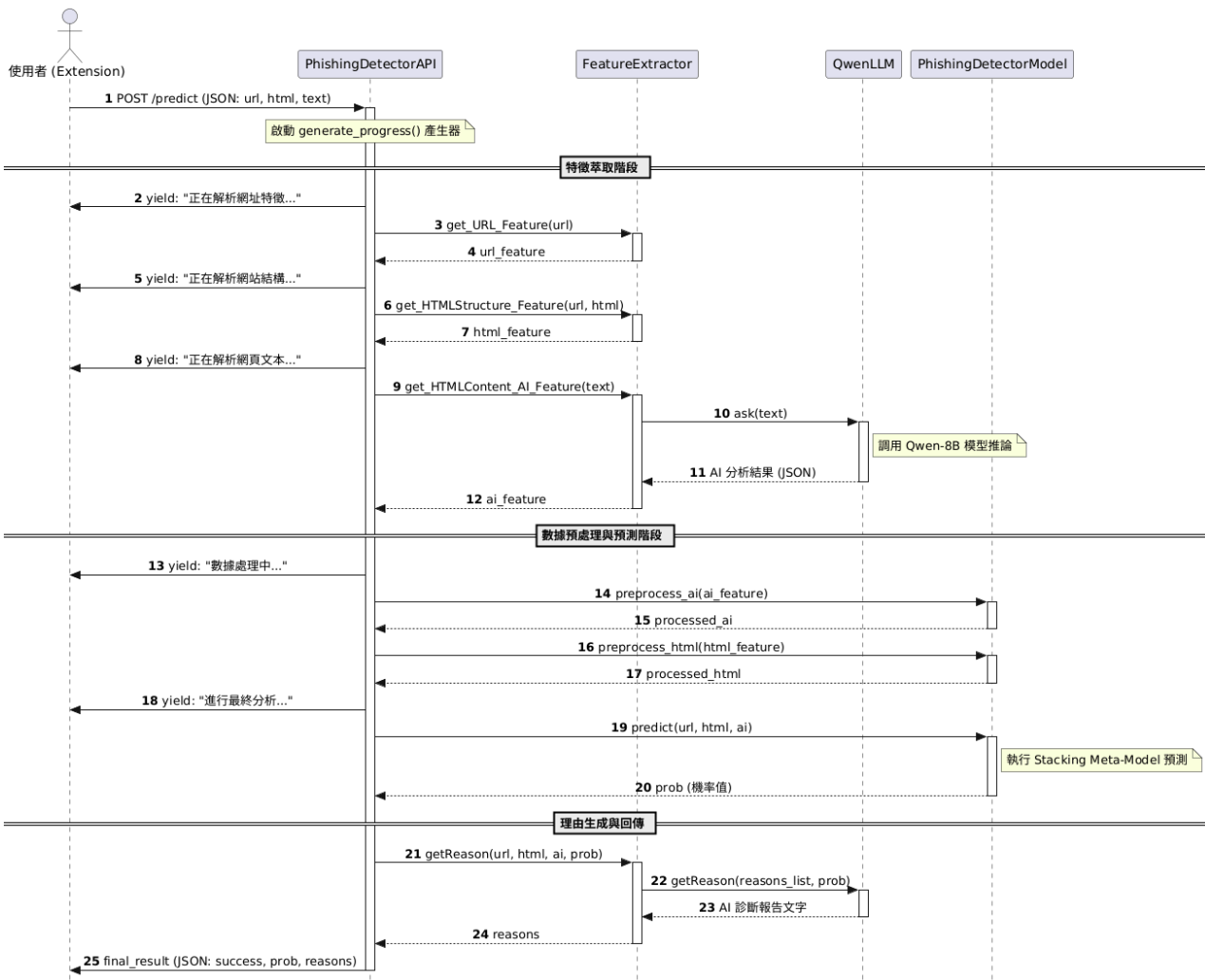
class QwenLLM:
    def __init__(self):
        self.model = Llama(
            model_path=r"D:\畢業專題\LLM\Qwen3-8B-Q4_K_S.gguf",
            n_gpu_layers=30,
            n_ctx=15000, #15000
            n_batch=2048, # 提升預處理速度
            n_ubatch=2048, # 同步提升運算效率
            flash_attn=True,
            use_mlock=True,
            type_k=8,
            type_v=8,
            verbose=False
        )
        self.system_prompt = self.system_prompt()
        self.max_retries = 1
        self.max_tokens=2048
        self.temperature=0.4
        self.response_format={"type": "json_object", "schema": PhishingSchema.model_json_schema()}
```

2. 釣魚網站辨識堆疊模型



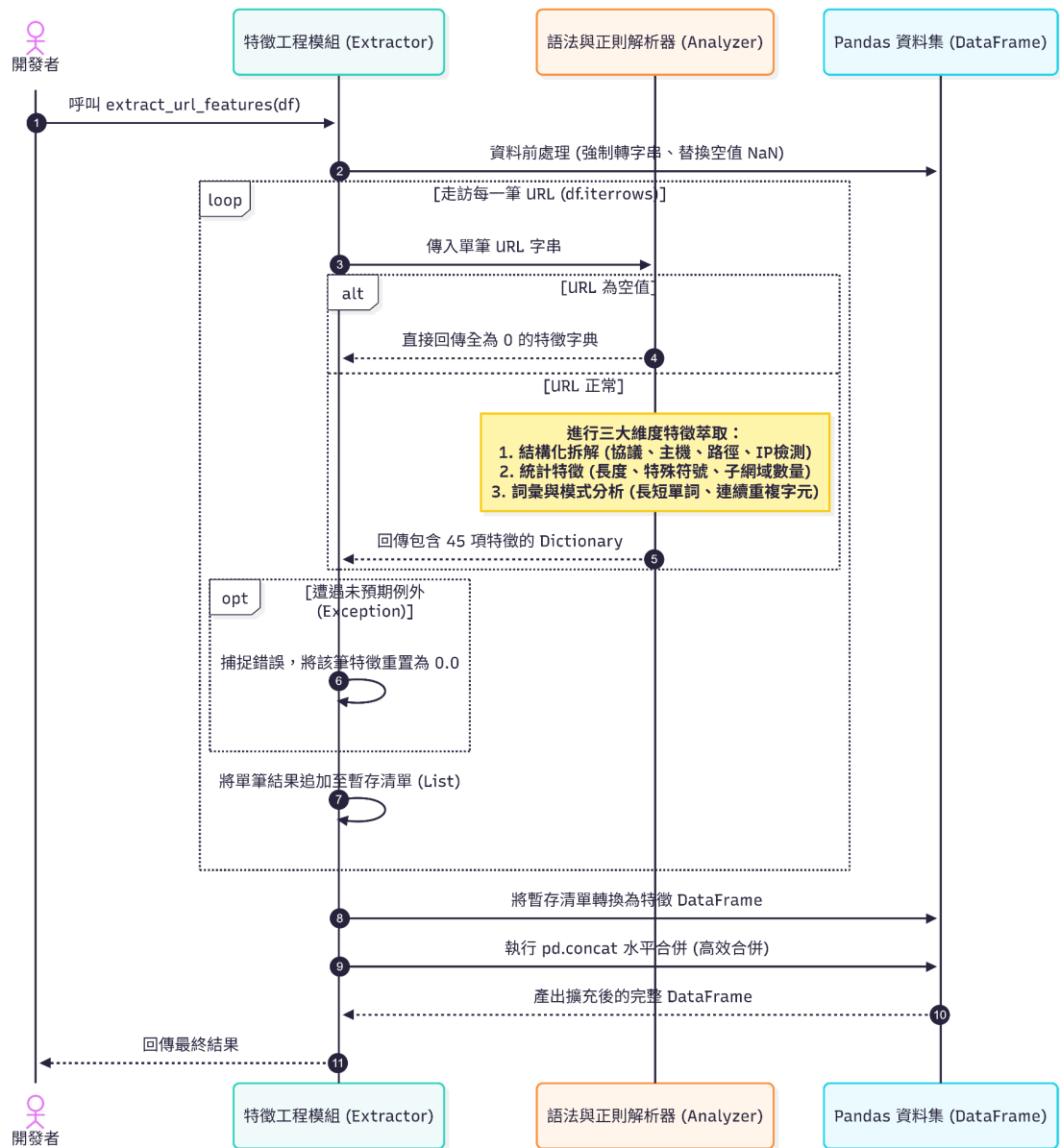
九、行為模型 (Behavioral Model)

9.1 系統循序圖 (應用階段)

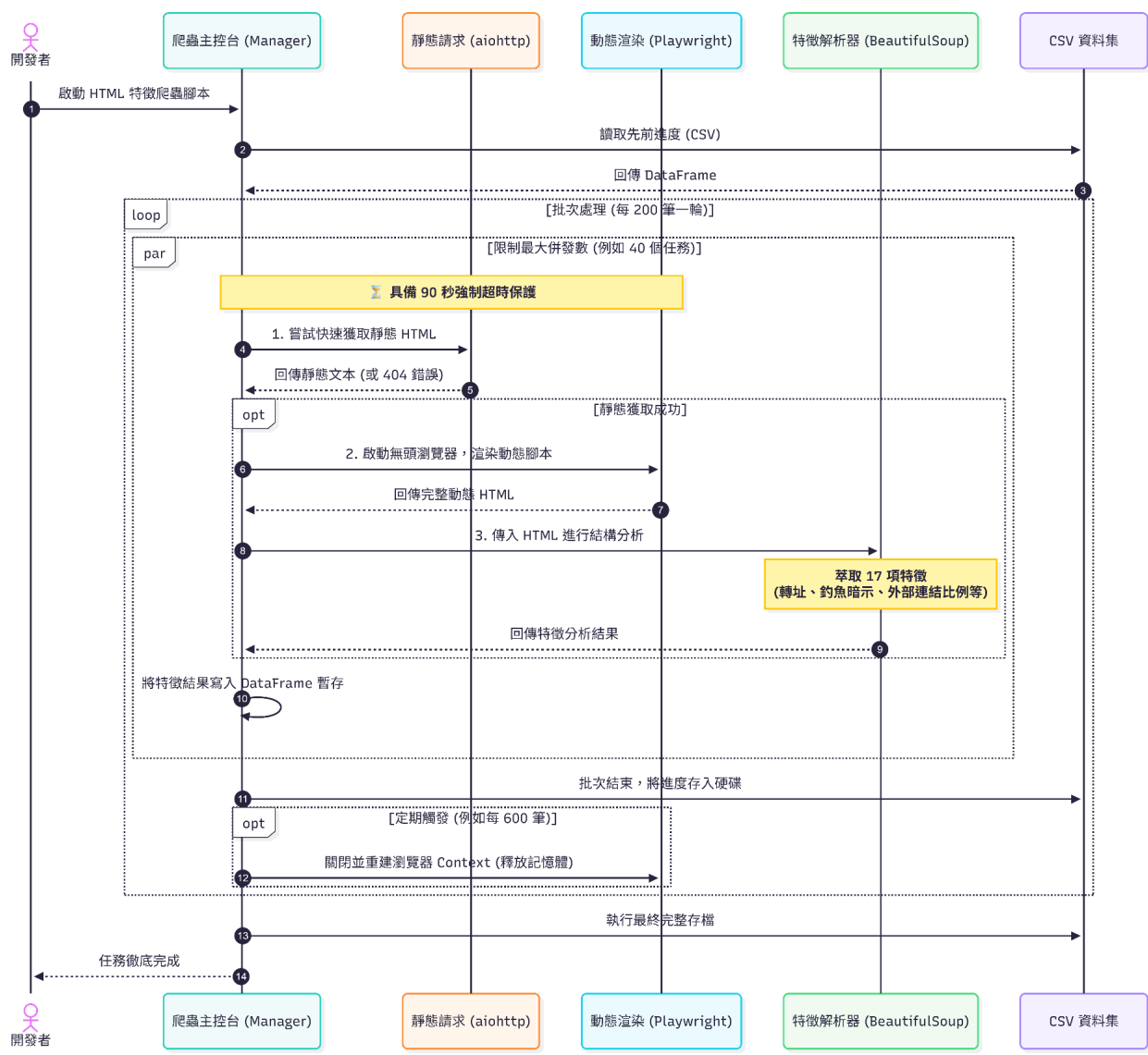


9.2 系統循序圖 (資料蒐集階段)

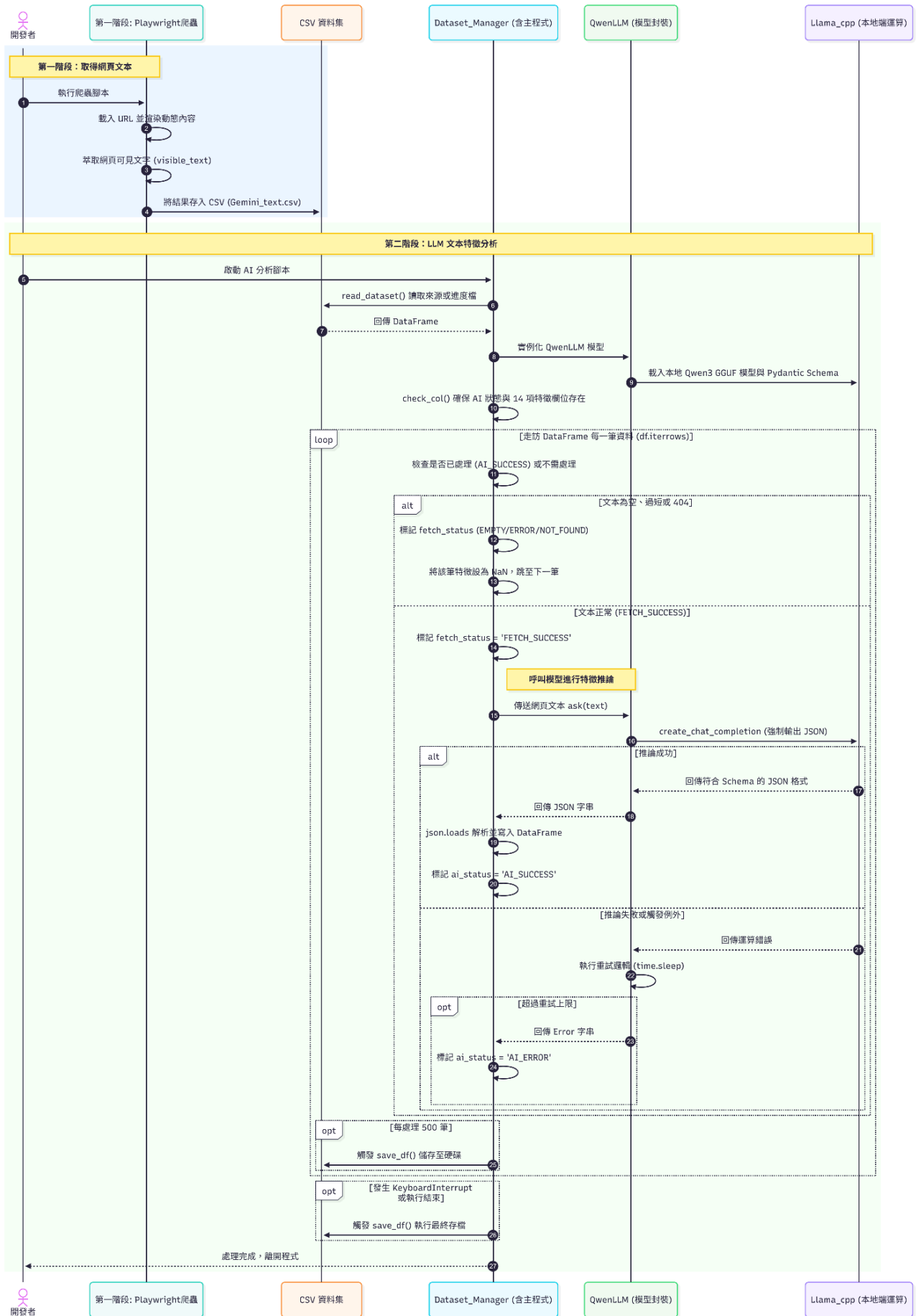
● URL特徵取得循序圖



● HTML特徵取得循序圖



● AI特徵取得循序圖



十、附錄

本附錄提供系統開發過程中的技術細節、實驗原始數據及環境配置說明，供研究者進行複現或深入技術參考。

10.1 本地端 SLM (Qwen-8B) 提示詞模板(語意特徵分析階段)

- System Prompt:

Role: Senior Cybersecurity Analyst (Phishing Detection)

Task

Strictly analyze the provided web text for phishing indicators. Produce a JSON assessment based ONLY on observable evidence.

Guidelines

1. **Objectivity**: Do not infer intent beyond what is explicitly written.
2. **Exclusivity**: Assessments must rely solely on the provided text string.
3. **Internal Logic**: Ensure the 'overall_phishing_likelihood_score' aligns with the boolean indicators (e.g., urgency + info request = high score).

Data Schema Definitions

- 'creates_urgency': (boolean) Does the text create a sense of urgency, pressure, or a deadline for the user to act quickly?
- 'uses_threats': (boolean) Does the text contain threats, warnings of account suspension, or other negative consequences if the user doesn't act?
- 'requests_sensitive_info': (boolean) Does the text explicitly or implicitly ask for login credentials (username, password), financial details (credit card numbers), or personal identification?
- 'offers_unrealistic_rewards': (boolean) Does the text offer prizes, lottery winnings, or other rewards that seem too good to be true?
- 'has_spelling_grammar_errors': (boolean) Are there noticeable spelling or grammatical errors that are uncharacteristic of a professional website?
- 'impersonated_brand': (string) What specific brand or company (e.g., "Facebook", "Google", "中華郵政") is the text trying to impersonate? This judgment must be based *solely* on mentions, logos, or design cues implied by the *text description* provided. If no specific brand is clear from the text, return "N/A".
- 'has_valid_copyright_year': (boolean) Does the footer or end of the text contain a standard copyright statement with a specific, recent year (e.g., "© 2024") and the brand name?
- 'is_content_login_focused': (boolean) Is the text content heavily saturated with keywords related to "login", "verification", "unlock account", or "update

password", while lacking general information about the company or service? (True = Overly focused on credential harvesting; False = Balanced content).

- 'has_rich_navigation': (boolean) Does the text contain evidence of a rich navigation structure, such as links to "About Us", "Careers", "Investor Relations", "Contact Support", "Press", or "Privacy Policy"? (True = Looks like a legitimate multi-page site; False = Looks like a single-page standalone form).

- 'has_physical_address': (boolean) Does the text clearly display a physical real-world address (e.g., street, city, country) for the organization?

- 'has_phone_number': (boolean) Does the text clearly display a customer support telephone number?

- 'content_consistency_score': (numeric, 0-10) Rate the semantic consistency between the different parts of the text (e.g., Title vs. Body, Brand Name vs. Footer Details).

- 0-3: Highly inconsistent. The page title says one thing (e.g., "Login") but the body text implies another, or the brand mentioned in the header does not match the copyright info.

- 4-6: Ambiguous or generic. Text is consistent but very vague (e.g., "Security Check" without specific brand details).

- 7-10: Highly consistent. The title, body content, footer, and contact info all align perfectly with a specific, identifiable organization's identity.

- 'language_professionalism_score': (numeric, 0-10) Rate the professionalism and grammatical correctness of the text.

- 0-2: Severely unprofessional, numerous glaring spelling/grammar errors, unclear language.

- 3-5: Noticeable errors, somewhat unprofessional, but understandable.

- 6-8: Mostly professional, minor or infrequent errors.

- 9-10: Highly professional, no discernible errors, official tone.

- Only assign scores of 0 or 10 if there is overwhelming, undeniable evidence to support such an extreme assessment. Most cases will fall between 2 and 8.

- 'overall_phishing_likelihood_score': (numeric, 0-10) Based on ALL textual clues (urgency, threats, info requests, rewards, language errors, impersonation, rich_navigation, phone, address), what is the overall likelihood that this is a phishing page?

- 0-2: Very low likelihood; text appears legitimate or benign.

- 3-5: Moderate likelihood; some suspicious elements, but not conclusive.
- 6-8: High likelihood; strong indicators of phishing.
- 9-10: Extremely high likelihood; undeniable phishing attempt based on text.
- Assign scores of 0 or 10 only if the evidence for or against phishing is absolute and unequivocal. Favor scores between 2 and 8 for nuanced cases.

IMPORTANT INTERNAL CHECK: Before generating the JSON, you MUST perform the following internal reasoning steps:

1. **Score Justification (Mental Step):** Silently (do not write this down) articulate to yourself *why* you are giving the `overall_phishing_likelihood_score` and `language_professionalism_score` their values. Base this reasoning *strictly* on the other fields you just filled out (e.g., "Score is 8 because `creates_urgency` is true, `uses_threats` is true, and `impersonated_brand` is 'Facebook'").
2. **Consistency Review:** Check if your mental justification from step 1 is logical and directly supported by the evidence in the text.
3. **Final Correction:** If you find any inconsistency (e.g., you gave a high score but all booleans are 'false'), go back and correct your scores *before* generating the final output.

- **User Prompt:**

Please analyze this text:/n {text}

- **text: 網頁文本內容**

10.2 本地端 SLM (Qwen-8B) 提示詞模板(AI評估生成階段)

- **System Prompt:**

你是一個冷酷的資安系統API，只負責輸出最終的報告文字。絕對禁止輸出任何思考過程、分析步驟、或『好的』、『首先』等對話用語。

- **User Prompt:**

請根據以下檢測數據，寫出約 100 字的繁體中文最終評估結論。

【檢測數據】

釣魚機率：{prob_percent}%

特徵分析：

{reasons_text}

【寫作指示】

1. {tone_instruction}

2. 直接給出結論，絕對不要包含「好的」、「首先」等對話用語。

- **reasons_text:根據網站特徵取得的要點清單**
- **tone_instruction(根據不同的機率值產生對應指示):**

- **<30%:**

該網站目前看來相對安全。請以安心、客觀的語氣解釋原因，告訴使用者可以放心瀏覽。

- **30%~70%:**

該網站存在部分可疑特徵，屬於中等風險。請提醒使用者保持警覺，不要隨意輸入密碼。

- **>70%:**

該網站具有極高的釣魚風險！請以強烈警告的語氣，強烈建議使用者切勿輸入任何資料。

10.3 模型訓練超參數配置

- **XGBoost (Level 0 - URL):**
 - n_estimators: 500
 - max_depth: 10
 - learning_rate: 0.029575
 - roc auc: 0.9959
- **XGBoost (Level 0 - HTML):**
 - n_estimators: 400
 - max_depth: 9
 - learning_rate: 0.017971
 - roc auc: 0.8785
- **XGBoost (Level 0 - AI):**
 - n_estimators: 500
 - max_depth: 9
 - learning_rate: 0.011045
 - roc auc: 0.8285
- **Logistic Regression (Level 1):**
 - C: 1.0, penalty: 'l2'

10.4 理由清單生成邏輯

為幫助系統評估風險指數之依據，理由清單直接根據三維特徵資料，並參考文獻與研究，人工制定可疑之項目清單。

特徵維度	判斷特徵	判斷依據	生成理由
URL	ip	是否使用 IP 隱藏真實網域	【網域異常】網址直接使用 IP 位址，這通常是為了規避網域審查的免洗惡意網站。
URL	punycode	檢查是否有同形異議字	【網域異常】網址使用了 Punycode (xn--) 編碼，企圖偽裝成正常字母以混淆視覺（同形異義字攻擊）。
URL	nb_at	檢查 @ 符號混淆	【網域異常】網址包含 '@' 符號，駭客常利用此特性隱藏真實的連線目標。
URL	nb_subdomains	大於三個子網域	【網域混淆】網址包含異常數量的子網域，企圖構造出極長的網址來掩蓋真實主機名稱。
URL	prefix_suffix	包含前後綴或二個以上的連字號	【網域混淆】網域包含可疑的連字號前後綴，企圖偽造官方品牌網域。
HTML	has_js_redirect	是否自動跳轉	【結構異常】網頁原始碼藏有自動跳轉指令，企圖在您不注意時導向隱蔽的惡意終端頁面。
HTML	ratio_extHyperlinks	超連結外部比例超過 0.6	【結構異常】網頁內含有大量指向外部網域的超連結，缺乏正規網站應有的內部連結架構。
HTML	empty_title	檢查標題與網域一致性	【身分不符】網頁標題空白或與所在網域名稱毫無關聯，缺乏品牌一致性。
HTML	phish_hints	檢查釣魚暗示詞	【高危險特徵】網頁原始碼中發現大量如 login、verify 等常見的釣魚誘餌關鍵字。
AI	impersonated_br	檢查品牌偽	【品牌冒用】AI 語意分析發現該網頁

	and	冒	企圖偽裝成知名品牌【{impersonated}】。
AI	creates_urgency	檢查威脅與急迫性	【社交工程】內文使用「急迫性」或「威脅性」話術（如帳號即將凍結），企圖迫使您倉促行動。
AI	requests_sensitive_info	檢查敏感資訊索取	【高危險行為】系統偵測到網頁正試圖誘導您輸入密碼、信用卡號等機密資訊。
AI	offers_unrealistic_rewards	檢查誘餌/不切實際的獎勵	【社交工程】內容提供不切實際的獎勵或優惠，利用貪心心理進行誘騙。
AI	has_spelling_grammar_errors	檢查專業度與文法	【專業度低】網頁內容存在明顯的拼寫或文法錯誤，這在嚴謹的官方網站中極少發生。
AI	has_physical_address	檢查聯絡資訊缺失	【來源可疑】網站缺乏實體地址與聯絡電話等正規商業網站應具備的聯絡資訊。

10.5 基模型調配演算法

本小節詳述基模型調配實驗所用之方法，藉由optuna選配模型最佳參數，並比較XGBoost、Fast SVM、MLP三個不同結構之模型

1. 使用StratifiedKFold進行驗證訓練、並根據標籤比例設定權重

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
ratio = float(np.sum(y == 0)) / np.sum(y == 1)
```

2. 根據需要進行標籤化或是標準化:

```
bool_col = ['creates_urgency', 'uses_threats', 'requests_sensitive_info',
            'offers_unrealistic_rewards', 'has_spelling_grammar_errors',
            'has_valid_copyright_year', 'is_content_login_focused',
            'has_rich_navigation', 'has_physical_address', 'has_phone_number']
df[bool_col] = df[bool_col].astype(int)
df['impersonated_brand'] = [1 if k else 0 for k in df['impersonated_brand'].notnull()]

num_cols = ['links_in_tags', 'nb_hyperlinks']
passthrough_cols = [col for col in X.columns if col not in num_cols]

preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), num_cols),
        ("pass", "passthrough", passthrough_cols)
    ]
)
```

3. XGBoost參數範圍設定:

```
def objective_xgb(trial):
    params = {
        "n_estimators": trial.suggest_int("n_estimators", 100, 600, step=100),
        "max_depth": trial.suggest_int("max_depth", 3, 15),
        "learning_rate": trial.suggest_float("learning_rate", 0.01, 0.2, log=True),
        "subsample": trial.suggest_float("subsample", 0.6, 1.0),
        "colsample_bytree": trial.suggest_float("colsample_bytree", 0.6, 1.0),

        # 固定參數
        "objective": "binary:logistic",
        "eval_metric": "auc",
        "tree_method": "hist",
        "scale_pos_weight": ratio,
        "random_state": 42
    }

    model = XGBClassifier(**params)
    scores = cross_val_score(model, X, y, cv=cv, scoring="roc_auc", n_jobs=-1)
    return scores.mean()

# 建立 Study 並尋找最佳解 (先測 20 組)
study_xgb = optuna.create_study(direction="maximize")
study_xgb.optimize(objective_xgb, n_trials=20)
```

4. Fast SVM參數範圍設定：

```
def objective_svm(trial):
    # 線性 SVM 的正則化強度 (類似 C 的倒數)
    alpha = trial.suggest_float("alpha", 1e-6, 1e-2, log=True)

    # 使用 SGDClassifier 實作線性 SVM，極速處理 14 萬筆資料
    svm = SGDClassifier(loss="hinge", penalty="l2", alpha=alpha, class_weight="balanced", random_state=42)

    # 透過 CalibratedClassifierCV 轉換出機率
    calibrated_svm = CalibratedClassifierCV(svm, method="sigmoid", cv=3)

    pipeline = Pipeline([
        ("preprocessor", preprocessor),
        ("svm", calibrated_svm)
    ])

    scores = cross_val_score(pipeline, X, y, cv=cv, scoring="roc_auc", n_jobs=-1)
    return scores.mean()

study_svm = optuna.create_study(direction="maximize")
study_svm.optimize(objective_svm, n_trials=30) # SVM 變快了，可以測多一點
```

5. MLP參數範圍設定：

```
def objective_mlp(trial):
    # 1. 動態組裝隱藏層：讓 Optuna 自己決定要 1~3 層，每層 32~256 個神經元
    n_layers = trial.suggest_int('n_layers', 1, 3)
    layers = []
    for i in range(n_layers):
        layers.append(trial.suggest_int(f'n_units_l{i}', 32, 256, step=32))
    hidden_layer_sizes = tuple(layers)

    # 2. 加入靈魂參數：學習率與 L2 正則化
    lr = trial.suggest_float("learning_rate_init", 1e-4, 1e-1, log=True)
    alpha = trial.suggest_float("alpha", 1e-5, 1e-1, log=True)

    mlp = MLPClassifier(
        hidden_layer_sizes=hidden_layer_sizes,
        learning_rate_init=lr,
        alpha=alpha,
        max_iter=300,
        early_stopping=True,
        random_state=42
    )

    # 3. 關鍵防漏水與抗不平衡：使用 ImbPipeline
    pipeline = ImbPipeline([
        ("preprocessor", preprocessor), # 你的標準化流程
        ("sampler", RandomUnderSampler(random_state=42)), # 強制將資料平衡為 1:1 給 MLP 看
        ("mlp", mlp)
    ])

    scores = cross_val_score(pipeline, X, y, cv=cv, scoring="roc_auc", n_jobs=-1)
    return scores.mean()

# 開始優化
study_mlp = optuna.create_study(direction="maximize")
# 建議給它至少 30~50 次的機會
study_mlp.optimize(objective_mlp, n_trials=30)
```

貳、系統規格書

內容目錄

- 一、系統技術架構 (Technical Architecture)
- 二、介面規格說明 (Interface Specification)
- 三、類別與方法 (Class & Method Design)
- 四、特徵工程實作規格 (Feature Engineering Spec)
- 五、系統使用流程 (System Usage Process)

一、系統技術架構 (Technical Architecture)

1.1 部署環境與連線機制

系統採用「客戶端控制、本地端運算」的邊緣運算模式，確保高隱私性與低延遲。

- **開發與執行環境**：Python 3.10+, JavaScript(ES6+)。
- **後端伺服器**：Flask 微服務框架，支援CORS跨域處理（允許Chrome Extension 調用）。
- **推論加速**：透過CUDA Toolkit實現llama-cpp-python的GPU代理，優化SLM推論效能。
- **網路拓樸**：前端插件與本地 API 之連線僅限於 localhost:5000。

二、介面規格說明 (Interface Specification)

2.1 偵測推論介面 (/predict)

本系統之核心通訊採用 NDJSON (Networked Data JSON) 串流響應格式，避免掃描期間前端因推論時間過長導致連線超時及使用者不安。

- Endpoint: `http://127.0.0.1:5000/predict`
- Method: POST
- Request Body (JSON):

```
const response = await fetch("http://127.0.0.1:5000/predict", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    url: tab.url,
    html: pageData.html,
    text: pageData.text
  })
});
```

- Response (application/x-ndjson):
 - `{ 'status': 'progress', 'message': '正在解析網址特徵...' }`
 - `{ 'status': 'progress', 'message': '正在解析網站結構...' }`
 - `{ 'status': 'progress', 'message': '正在解析網頁文本...' }`
 - `{ 'status': 'progress', 'message': '數據處理中...' }`
 - `{ 'status': 'progress', 'message': '數據處理完成!' }`
 - `{ 'status': 'progress', 'message': '進行最終分析...' }`
 - `{ 'status': 'progress', 'message': '最終分析完成!' }`
 - `{ "status": "success", "message": prob[0], "reasons": reasons }`

註:prob[0]代表風險指標，reasons為判斷理由

三、類別與方法 (Class & Method Design)

3.1 專案樹狀目錄結構(本地)

Phishing-Detection-System/

- |—— LLM/
 - | |—— Qwen3-8B-Q4_K_S.gguf
- |—— phishing-backend/
 - | |—— model/
 - | | |—— html_mlp_v1.keras
 - | | |—— meta_model_logistic_v1.pkl
 - | | |—— optuna_xgb_ai_final.json
 - | | |—— optuna_xgb_html_final.json
 - | | |—— optuna_xgb_url_final.json
 - | |—— app.py
 - | |—— FeatureExtractor.py
 - | |—— phishing_detector_model.py
- |—— phishing-detector-extension/
 - | |—— icon.jpg
 - | |—— icon.png
 - | |—— manifest.json
 - | |—— popup.html
 - | |—— popup.js
- |—— AI_feature.py
- |—— prompt3.txt

3.2 專案樹狀目錄結構(colab)

colab/

- |—— 元模型建立/
 - | └—— composite_model.ipynb
- |—— 創建資料集/
 - | |—— selenium_crawler.ipynb
 - | |—— HTML_play_wright.ipynb
 - | |—— 創建非AI欄位URL.ipynb
 - | |—— 製作終極資料集.ipynb
 - | └—— 重新切割資料集.ipynb
- |—— 建立基模型/
 - | |—— AI_model/
 - | | └—— 終極版AI模型製作.ipynb
 - | |—— HTML_model/
 - | | └—— 終極版HTML模型製作.ipynb
 - | └—— URL_model/
 - | | └—— Model_URL_optuna.ipynb

3.3 檔案與負責項目

檔案	負責項目
Qwen3-8B-Q4_K_S.gguf	邊緣語言模型
optuna_xgb_ai_final.json	最終版之AI語意特徵基模型權重(Level 0)
optuna_xgb_html_final.json	最終版之HTML語意特徵基模型權重(Level 0)
optuna_xgb_url_final.json	最終版之URL語意特徵基模型權重(Level 0)
meta_model_logistic_v1.pkl	最終版堆疊模型之元模型(Level 1)
app.py	前後端API之主要接口
FeatureExtractor.py	特徵萃取與管理語言模型
phishing_detector_model.py	實作堆疊模型與模型預測
icon.jpg/png	擴充套件之icon
manifest.json	定義擴充套件與權限宣告
popup.html	定義插件UI介面
popup.js	定義插件前端邏輯與資料流管理
AI_feature.py	用於模型訓練階段，將網頁文本結合prompt並透過SLM生成AI網頁文本特徵
prompt3.txt	SLM系統提示詞，可參考系統計畫書之附錄
composite_model.ipynb	結合三個基模型訓練生成元模型
selenium_crawler.ipynb	playwright爬蟲取得網頁文本
HTML_playwright.ipynb	playwright爬蟲取得網頁結構，並分析生成網頁結構特徵
創建非AI欄位URL.ipynb	分析URL網址取得網頁網址特徵
製作終極資料集.ipynb	取得三個特徵資料集之交集資料集
重新切割資料集.ipynb	拆出分別訓練三個基模型之三個特徵資料集與元模型訓練資料集
終極版AI模型製作.ipynb	網頁文本特徵模型比較與權重配置
終極版HTML模型製作.ipynb	網頁結構特徵模型比較與權重配置
終極版URL模型製作.ipynb	網頁網址特徵模型比較與權重配置

3.4 類別與方法說明

Class Name: PhishingDetectorAPI	ID: 1
Purpose: 1. 系統之主入口與主控制器，負責管理前端網頁服務(Flask web Server)的生命週期。 2. 接收前端擴充功能發送的網頁原始數據，排成特徵萃取控制，並利用NDJSON串流技術回饋即時系統進度與最終報告至客戶端。	
Attributes: 1. app: Flask實例，建立與管理Web api服務。 2. detector: PhishingDetectorModel實例，負責 AI 模型推論與數據預處理。 3. extractor: FeatureExtractor實例，負責執行三路多模態特徵萃取。 4. url: 儲存由請求端傳入的原始網址。 5. html_structure: 儲存由請求端傳入的網頁結構(DOM)。 6. html_content: 儲存由請求端傳入的純文本數據。 7. url_feature: 儲存經萃取後尚未輸入模型前的網頁網址特徵向量。 8. html_feature: 儲存經萃取後尚未輸入模型前的網頁結構特徵向量。 9. ai_feature: 儲存經萃取後尚未輸入模型前的網頁文本特徵向量。	
Method Names:	Function Description:
__init__()	初始化應用程式並配置CORS跨域資源共享。調用內部方法設置路由並實例化系統主模型模組與特徵萃取模組。
_setup_routes()	定義API路由，將端點(/predict)映射至方法(predict())，並設定POST與OPTIONS請求。
predict()	系統核心推論調度方法。負責解析JSON請求封包，包裝其子方法之結果為Response物件並回傳。
generate_progress()	predict()的內部子方法，透過yield實現非同步進度回饋，其執行順序包含：擷取URL/HTML/AI特徵、執行數據前處理、執行Stacking模型預測、產出診斷理由、回傳成功封包。以上方法藉由detector與extractor完成實現。具備除錯機制確保後端異常能轉換為錯誤包封回傳。
run()	啟動本地伺服器連線，實施監聽。

Class Name: FeatureExtractor	ID: 2
Purpose: 1. 特徵萃取模組，負責從網址、DOM、網頁文本根據資料字典萃取出特徵資料。 2. 管理系統中之語言模型使用，包含特徵分析與生成評估。	
Attributes: 1. llm: QwenLLM實例，負責呼叫語言模型供系統使用。	
Method Names:	Function Description:
__init__()	初始化，建立語言模型實例。
get_URL_Feature()	根據資料字典實作出網頁網址特徵。
get_HTMLStructure_Feature()	根據資料字典實作出網頁結構特徵。
get_HTMLContent_AI_Feature()	根據資料字典實作出網頁文本特徵。
is_not_found_page()	判斷網站是否屬於404錯誤的網頁
getReason()	結合理由清單與風險指數並傳入語言模型獲取評估結果。
get_reason_list()	藉由三維之特徵並根據「系統計畫書-附錄-理由清單生成邏輯」建立理由清單。

Class Name: PhishingDetectorModel	ID: 3
Purpose: 1. 系統主模型模組。 2. 負責模型預測、堆疊模型實作。 3. 包含前處理與檢查。	
Attributes: 1. URL_Feature_Model: 存放Level 0之URL模型。 2. HTMLStructure_Feature_Model: 存放Level 0之HTML模型。 3. HTMLContent_AI_Feature_Model: 存放Level 0之AI模型(文本分析模型)。 4. Meta_Model: 存放Level 1之元模型。 5. feature_vector: 存放 Level 0模型預測結果組合之特徵向量。	
Method Names:	Function Description:
__init__()	初始化，透過呼叫方法取載入模型檔案。
predict()	外部使用者呼叫的方法，負責依序呼叫其他方法完成堆疊模型預測。
check_feature()	檢查URL、HTML以及AI特徵缺漏或有誤。
get_URL_Feature_Model()	讀取並載入URL模型檔案。
get_HTMLStructure_Feature_Model()	讀取並載入HTML模型檔案。
get_HTMLContent_AI_Feature_Model()	讀取並載入AI模型檔案。
get_Meta_Model()	讀取並載入元模型檔案。
get_feature_vector()	回傳Level 0組合之特徵向量。
set_feature_vector()	將三路特徵傳入基模型訓練並將各自輸出結合成新特徵向量。
preprocess_html()	前處理HTML特徵，包含刪除除錯欄位。
preprocess_ai()	前處理AI特徵，包含標籤轉換。

Class Name: QwenLLM	ID: 4
Purpose: - 系統中使用之語言模型。 - 版本為開源之Qwen3-8B-Q4_K_S.gguf。	
Attributes: - model: 語言模型之相關設定(llama管理)，如:model_path、n_gpu_layers、n_ctx、n_batch、n_ubatch、flash_attn、use_mlock、type_k、type_v、verbose等。 - system_prompt: 儲存特徵分析需要的系統提示詞。 - max_retries: 設定最大重試次數。 - max_tokens: 設定最大回應token數。 - temperature: 設定語言模型溫度(回應自由度)。 - response_format: 設定模型遵守json型態，格式依照PhishingSchema。	
Method Names:	Function Description:
__init__()	初始化，透過呼叫方法取得系統提示詞。
system_prompt()	讀取提示詞所在之文件檔並回傳。
ask()	用於文本分析，輸入文本得到特徵回應。
getReason()	根據理由清單與風險指數，並搭配自定義提示詞(參考系統計畫書之10.2)取得AI評估結果。

Class Name: PhishingSchema	ID: 5
Purpose: 定義語言模型對於網頁文本特徵所固定輸出之特徵與格式	
Attributes: 參考系統計畫書之附錄	

四、核心模組實作規格 (Core Module Implementation Specification)

4.1 前端插件採集與權限模組 (Frontend Collection Module)

本模組負責在使用者觸發掃描時，安全地從當前瀏覽分頁中擷取原始數據。

- 權限約束 (Manifest V3):

- scripting: 授權系統執行chrome.scripting.executeScript，將採集邏輯動態注入目標頁面。
- activeTab: 確保採集行為僅在使用者主動啟動插件時觸發，符合最小權限原則。
- host_permissions: 宣告http://127.0.0.1:5000/*為唯一合法之數據傳輸目標。

```
"permissions": [  
  "activeTab",  
  "scripting",  
  "tabs"  
],
```

- 數據採集邏輯:

- 使用document.documentElement.outerHTML擷取完整 DOM 結構。
- 使用document.body.innerText擷取可見文本，並執行初步清洗以去除無效空白。

```
return {  
  html: document.documentElement.outerHTML,  
  text: document.body.innerText  
};
```

4.2 後端 API 串流與調度模組 (Backend Stream Controller)

本模組負責管理長時間推論過程中的通訊穩定性。

- **串流機制**：利用Flask Response物件封裝Python生成器，將mimetype設為application/x-ndjson。

```
return Response(generate_progress(), mimetype='application/x-ndjson')
```

- **狀態監控**：定義五大關鍵進度節點（網址、結構、文本、預處理、最終推論），每完成一階段即yield一筆JSON數據至前端。

```
yield json.dumps({'status': 'progress', 'message': '正在解析網址特徵...'}) + '\n'
self.url_feature = self.extractor.get_URL_Feature(self.url)
yield json.dumps({'status': 'progress', 'message': '正在解析網站結構...'}) + '\n'
self.html_feature = self.extractor.get_HTMLStructure_Feature(self.url, self.html_structure)
yield json.dumps({'status': 'progress', 'message': '正在解析網頁文本...'}) + '\n'
self.ai_feature = self.extractor.get_HTMLContent_AI_Feature(self.html_content)
yield json.dumps({'status': 'progress', 'message': '數據處理中...'}) + '\n'
self.ai_feature = self.detector.preprocess_ai(self.ai_feature)
self.html_feature = self.detector.preprocess_html(self.html_feature)
yield json.dumps({'status': 'progress', 'message': '數據處理完成!'}) + '\n'
yield json.dumps({'status': 'progress', 'message': '進行最終分析...'}) + '\n'
prob = self.detector.predict(self.url_feature, self.html_feature, self.ai_feature)
print(prob)
reasons = self.extractor.getReason(self.url_feature, self.html_feature, self.ai_feature, prob[0])
print(f'reason:{reasons}')
yield json.dumps({'status': 'progress', 'message': '最終分析完成!'}) + '\n'
final_result = {
    "status": "success",
    "message": prob[0],
    "reasons": reasons
}
yield json.dumps(final_result) + '\n'
```

- **異常隔離**：實作全部採取try-except機制，若特徵萃取失敗（如SLM逾時），系統須 Yield 錯誤包並優雅關閉連結。

```
except Exception as e:
    yield json.dumps({"status": "error", "message": str(e)}) + "\n"
```

4.3 多模態特徵工程實作 (Multimodal Feature Engineering)

本模組為系統之運算核心，負責將原始輸入轉化為 76 維特徵向量。

- URL/結構特徵 (Dim 0-60):

- 採用正則表達式進行 45 項網址統計。
- 利用BeautifulSoup(lxml解析器)執行16項DOM分析。

```
results = []
features = {}
word_split_pattern = re.compile(r'^a-zA-Z0-9+')

parsed_url = urlparse(url)
scheme = parsed_url.scheme
hostname = parsed_url.hostname if parsed_url.hostname else ''
path = parsed_url.path
query = parsed_url.query
fragment = parsed_url.fragment

soup = BeautifulSoup(html, 'html.parser')
parsed_url = urlparse(url)
```

- 語意特徵路徑 (Dim 61-75):

- 呼叫本地Qwen-8B (GGUF 4-bit)執行15項心理語意標記。

```
response = self.model.create_chat_completion(
    messages=[
        {"role": "system", "content": self.system_prompt},
        {"role": "user", "content": f"Please analyze this text:\n {text}"}
    ],
    max_tokens=self.max_tokens,
    temperature=self.temperature,
    response_format=self.response_format
)
return response["choices"][0]["message"]["content"]
```

- 定義AI返回之評分(0-10)轉化邏輯，確保語意特徵具備數學代表性(參考系統計畫書附錄10.1)。

4.4 堆疊模型與推論整合模組 (AI Stacking & Inference)

本模組執行最終的風險決策融合。

- **預處理對齊：**載入特徵進行欄位檢查並適時標籤編碼(因使用XGBoost不使用標準化)，確保輸入模型後沒有問題。
- **兩層式推論：**
 - Level 0: 呼叫 XGBoost (URL/HTML/Text) 產生初階預測機率。

```
self.feature_vector = np.c_[self.URL_Feature_Model.predict_proba(url_feature)[: , 1],
                             self.HTMLStructure_Feature_Model.predict(html_feature).ravel(),
                             self.HTMLContent_AI_Feature_Model.predict_proba(ai_feature)[: , 1]]
```

- Level 1: 將 L0 輸出送入 Logistic Regression 模型產出最終風險機率指數。

```
prob = self.Meta_Model.predict_proba(self.feature_vector)[: , 1]
return prob
```

- **理由生成：**根據預測分數與高貢獻度特徵，調用extractor.getReason產出繁體中文診斷報告提供客戶理解性。

```
prob_percent = round(probability * 100, 2)

try:
    response = self.model.create_chat_completion(
        messages=[
            {"role": "system", "content": system_content},
            {"role": "user", "content": prompt}
        ],
        max_tokens=500,
        temperature=self.temperature,
    )
    raw_text = response["choices"][0]["message"]["content"]
    cleaned_text = re.sub(r'<think>.*?</think>', '', raw_text, flags=re.DOTALL).strip()
    return cleaned_text
except Exception as e:
    print('理由生成錯誤')
    return None
```

(詳細參數參考系統計畫書附錄10.2)

4.5 自動化資料蒐集與預處理 (Automated Data Collection & Preprocessing)

本節詳述系統研究開發階段中，「自研資料集」之採集算法與處理工序。

4.5.1 樣本來源與獲取策略

樣本來源為 Kaggle 與 NIST Phish Scale 內別人已經整理好之網址資料。

4.5.2 URL 特徵自動化萃取演算法

為提升特徵萃取與資料蒐集速度，本專案開發自動化萃取演算法，旨在將原始網址字串轉化為具備 45 個維度的統計向量(資料字典內有更詳細之說明)。

1. **組件分解**：使用 `urlparse` 拆解 URL。

```
parsed_url = urlparse(url)
hostname = parsed_url.hostname if parsed_url.hostname else ''
```

2. **指標計算**：統計符號頻率、IP 檢測、Punycode 與DGA隨機性偵測(數字佔比)。

```
# 4. nb_dots (點號數量)
features['nb_dots'] = hostname.count('.')
```

3. **詞彙統計**：分析主機名與路徑中單詞的最短、最長與平均長度。

```
# 38. shortest_word_host: 主機名稱中最短單詞的長度。
features['shortest_word_host'] = min(len(word) for word in hostname_words) if hostname_words else 0
```

4.5.3 HTML 結構特徵蒐集演算法

本專案實作了一套基於「非同步」的網頁採集框架。

1. **併發與效能優化**：利用`asyncio.Semaphore`限制併發數，並透過資源阻斷技術攔截媒體載入。

```
CONCURRENCY_LIMIT = 20
BATCH_SIZE = 200
RESTART_INTERVAL = 200
RENDER_WAIT_TIME = 2

# 建立號誌，限制同時執行的任務數量
semaphore = asyncio.Semaphore(CONCURRENCY_LIMIT)

task = asyncio.create_task(
    safe_worker_wrapper(index, row, session, context, df, html_feature_columns, total_rows)
)
tasks.append(task)
```

2. **雙引擎策略**：結合[aiohttp](#)靜態快取與[Playwright](#)動態渲染。

```
async with aiohttp.ClientSession(connector=aiohttp.TCPConnector(ssl=False)) as session:
    async with async_playwright() as p:
        browser = await p.chromium.launch(headless=True, args=['--no-sandbox', '--disable-dev-shm-usage'])
        context = await browser.new_context(ignore_https_errors=True)

        try:
            response = await page.goto(url, timeout=20000, wait_until='domcontentloaded')
        except PlaywrightError as e:
            if "interrupted by another navigation" in str(e):
                try: await page.wait_for_load_state('domcontentloaded', timeout=20000)
                except: pass
            else:
                raise e

        await page.wait_for_timeout(RENDER_WAIT_TIME * 1000)

        # 簡單滾動一下
        await page.evaluate("window.scrollTo(0, document.body.scrollHeight / 2);")
        await page.wait_for_timeout(500)
```

3. **特徵實作邏輯**：包含身份一致性、資源關聯性、行為異常偵測與安全錨點分析。

```
df.at[index, 'ratio_intHyperlinks'] = int_links / total_links if total_links > 0 else 0.0
df.at[index, 'ratio_extHyperlinks'] = ext_links / total_links if total_links > 0 else 0.0
df.at[index, 'ratio_extRedirection'] = red_count / len(all_links) if all_links else 0.0
df.at[index, 'ratio_extErrors'] = err_count / len(all_links) if all_links else 0.0
```

4. 輸出畫面：

```
🔍 處理中 [9792/140978]: http://www.simple.urlfb.co
🔍 處理中 [9793/140978]: https://closure-floridacu.web.app/
🔍 處理中 [9794/140978]: https://www.fpu.ac.jp
🔍 處理中 [9795/140978]: http://www.bmq-mellat-ir.gq
🔍 處理中 [9798/140978]: https://saclpiu7y.firebaseio.com/
🔍 處理中 [9799/140978]: http://www.a.am
🔍 處理中 [9800/140978]: https://www.southerngaragebands.com

💾 --- 已處理至 9800 筆，寫入 Google Drive 中... ---
♻️ 達到重啟門檻，重建瀏覽器 Context 釋放深層記憶體...
🔍 處理中 [9802/140978]: https://www.googlefight.com
🔍 處理中 [9801/140978]: https://mail-104425.weeblysite.com/
🔍 處理中 [9809/140978]: https://www.climatecolab.org
🔍 處理中 [9812/140978]: http://www.antycenzor.pl
```

4.5.4 AI 文本語意特徵標註演算法

本專案之語意標註流程分為「網頁文本採集」與「SLM推理」兩階段。

1. 動態文本擷取技術：

- **轉址容錯機制：**在執行爬蟲時監聽指定錯誤，確保轉址後獲取真實文本。

```
try:
    response = await page.goto(url, timeout=30000, wait_until='domcontentloaded')
except PlaywrightError as e:
    # 如果錯誤訊息包含 "interrupted"，表示發生了頁面跳轉
    if "interrupted by another navigation" in str(e):
        print(f" [Info] 偵測到轉址 (Redirect)，正在等待新頁面載入: {url}")
        # 等待新頁面載入完成 (這很重要，不然會抓到空內容)
        try:
            await page.wait_for_load_state('domcontentloaded', timeout=30000)
        except:
            pass # 如果等待超時，就盡量抓現有的
        response = None # 轉址後 response 物件可能不準確，設為 None
    else:
        # 如果是其他錯誤 (如連線失敗)，則正常拋出
        raise e
```

- **視覺化採集：**使用inner_text("body")模擬人類閱讀視角，產出純淨語意樣本。

```
text = await page.inner_text("body")
```

2. SLM推理與結構化輸出：本階段利用Qwen-8B進行心理學層面的偵測。

- **結構化 Schema 規範**：基於定義之PhishingSchema(系統計畫書8.3模型架構)，強制模型針對急迫感、威脅感、敏感資訊請求、品牌冒用等 15 項心理特徵進行JSON格式化輸出。

```
self.response_format={"type": "json_object", "schema": PhishingSchema.model_json_schema()}}
```

- **專家啟發式過濾**：實作「網頁不存在」判斷邏輯，偵測文本中是否包含「404」、「頁面不存在」等關鍵字，自動過濾掉無效樣本，確保訓練集標記品質。

```
def is_not_found_page(self, text):
    not_found_patterns = [
        "404",
        "the page you are looking for does not exist",
        "找不到頁面",
        "找不到網頁",
        "404 - 找不到頁面",
        "頁面不存在",
        'not found',
        'page not found',
        'page does not exist',
        'site not found',
        'no tenant found'
    ]
    text_lower = text.lower()
    count = sum(kw in text_lower for kw in not_found_patterns)
    return count >= 2
```

- **邊緣推論參數優化**：配置 n_gpu_layers=30 將運算委派至 GPU；設定 n_ctx=15000 以處理長篇網頁文本；並開啟 flash_attn 與 mlock 提升 KV 快取效率。

```
self.model = Llama(
    model_path=r"D:\畢業專題\LLM\Qwen3-8B-Q4_K_S.gguf",
    n_gpu_layers=30,
    n_ctx=15000,
    n_batch=2048, # 提升預處理速度
    n_ubatch=2048, # 同步提升運算效率
    flash_attn=True,
    use_mlock=True,
    type_k=8,
    type_v=8,
    verbose=False
)
self.system_prompt = self.system_prompt()
self.max_retries = 1
self.max_tokens=2500
self.temperature=0.4
```

3. 標註流水線與異常控制：

- **狀態驅動管理**：由Dataset_Manager監控ai_status與fetch_status，實現自動斷點續傳。

```
print(f'正在處理第{index+1}/{len(df_process)}筆資料, url:{row["url"]}')
if row['ai_status'] == 'AI_SUCCESS' and pd.notna(row['creates_urgency']):
    print(f'已經處理過，跳過')
    continue

elif text.startswith('FETCH_ERROR:'):
    print('fetch_status為FETCH_ERROR，跳過')
    df_process.loc[index, 'fetch_status'] = 'FETCH_ERROR'
    df_process.loc[index, self.col] = np.nan
    continue

elif text.startswith('FETCH_EMPTY:'):
    print('fetch_status為FETCH_EMPTY，跳過')
    df_process.loc[index, 'fetch_status'] = 'FETCH_EMPTY'
    df_process.loc[index, self.col] = np.nan
    continue
```

- **自動化重試機制**：針對 LLM 生成失敗之樣本，實作重試邏輯。

```
except Exception as e:
    ex = e
    print("❌ 錯誤：", e)
    wait = (attempt+1) * 1
    print(f"等待 {wait} 秒後重試...")
    time.sleep(wait)
```

4.6 多模態數據整合與 Stacking 訓練集切割 (Data Integration & Splitting)

本節定義如何將獨立採集的三路特徵模態進行精確對齊，並構建用於兩層式 Stacking 模型訓練的高品質資料集。

4.6.1 資料清洗與主索引一致化

在進行多模態整合前，必須解決不同採集腳本間可能產生的字串不一致問題。透過 `strip()` 與 `drop_duplicates` 確保網址索引之唯一性。

```
def clean_df(df, name):
    # 確保 URL 是字串，去除前後空白，避免因為多了一個空格而對不起來
    df['url'] = df['url'].astype(str).str.strip()

    # 處理重複：如果同一個網址在特徵表出現兩次，會導致合併後筆數爆炸
    original_len = len(df)
    df = df.drop_duplicates(subset=['url'], keep='first')
    if len(df) < original_len:
        print(f"提醒：{name} 發現重複 URL，已移除 {original_len - len(df)} 筆。")
    return df
```

正在讀取資料集...

執行資料清洗...

提醒：URL主表 發現重複 URL，已移除 470 筆。

提醒：HTML表 發現重複 URL，已移除 451 筆。

提醒：AI表 發現重複 URL，已移除 470 筆。

開始合併... 初始主表筆數：246755

4.6.2 多模態交集提取與特徵淨化

系統採用「嚴格交集」策略，僅保留三路特徵接成功取得之樣本並移除用於流程控制之非特徵欄位，再次觀察資料標籤分布。

```
df_final = df_final[df_final['ai_status'] == 'AI_SUCCESS']
df_final = df_final[df_final['feature_extracted'] == 1]
df_final.drop('ai_status', axis=1, inplace=True)
df_final.drop('feature_extracted', axis=1, inplace=True)
df_final.drop('fetch_status', axis=1, inplace=True)
```

=== 🎨 Target 數量分布 ===

target	
1	96204
0	12975

=== 📊 Target 比例分布 (%) ===

target	
1	88.12
0	11.88

4.6.3 Stacking 層級訓練集切割與規範

為了確保兩層式模型架構的泛化能力，本系統實作了資料隔離機制，防止 Level 1 元模型產生資料過擬合。

1. **分層抽樣拆分：** 從「交集資料集」中執行分層抽樣（stratify=target），固定分配 50% 之完整特徵資料作為Meta_Set。此子集專用於產出Level 0的預測機率，並作為 Level 1 元模型訓練與最終驗證之依據。

```
df_base_pool, df_meta_set = train_test_split(
    df_intersection,
    test_size=0.50, # 50% 給元模型
    stratify=df_intersection['target'],
    random_state=42 # 設定亂數種子，確保每次重跑結果都一樣
)
```

2. **基模型隔離策略：** 為了防止 Level 0 基礎模型在訓練時「看過」元模型的驗證數據，系統實作了反向過濾機制。

- **技術實作：** 將 Meta_Set 之網址轉換為 Set 集合對象（提高檢索效率），並利用 ~isin() 運算，從 URL、HTML、AI 的各模態原始資料中，徹底排除與 Meta_Set 重疊之網址

```
meta_urls_forbidden = set(df_meta_set['url'].tolist())

df_url_train = df_url_raw[~df_url_raw['url'].isin(meta_urls_forbidden)]
df_html_train = df_html_raw[~df_html_raw['url'].isin(meta_urls_forbidden)]
df_ai_train = df_ai_raw[~df_ai_raw['url'].isin(meta_urls_forbidden)]
```

3. **模型效能驗證資料及切割：** 從Meta_Set中再切出20%作為最後測試集並用來驗證堆疊效能。

```
from sklearn.model_selection import train_test_split
df_meta_train, df_meta_test = train_test_split(
    df,
    test_size=0.2,
    stratify=df['target'],
    random_state=42
)
df_meta_test.to_csv('meta_test.csv', index=False)
```

4.6.4 數據分佈與標籤平衡觀察

檢視分割之資料集標籤分布情形與資料集大小，利於後續訓練進行不平衡標籤處理。

👉 執行分層抽樣拆分 (維持 Target 比例)...

✓ Meta_Set (元模型專屬): 54590 筆

✓ Base_Pool (交集剩餘): 54589 筆

從基模型資料集中剔除 Meta_Set 網址...

✓ URL 基模型訓練集: 從 247225 筆 ➡ 縮減為 192611 筆

✓ HTML 基模型訓練集: 從 125618 筆 ➡ 縮減為 71005 筆

✓ AI 基模型訓練集: 從 135646 筆 ➡ 縮減為 81036 筆

📊 驗證 基模型訓練集 中的 Target 比例:

=====URL=====

target

0 52.0

1 48.0

Name: proportion, dtype: float64

=====HTML=====

target

1 82.42

0 17.58

Name: proportion, dtype: float64

=====AI=====

target

1 81.61

0 18.39

📊 驗證 Meta_Set 中的 Target 比例:

target

1 88.12

0 11.88

4.7 機器學習模型訓練與堆疊建置 (Model Training & Stacking Construction)

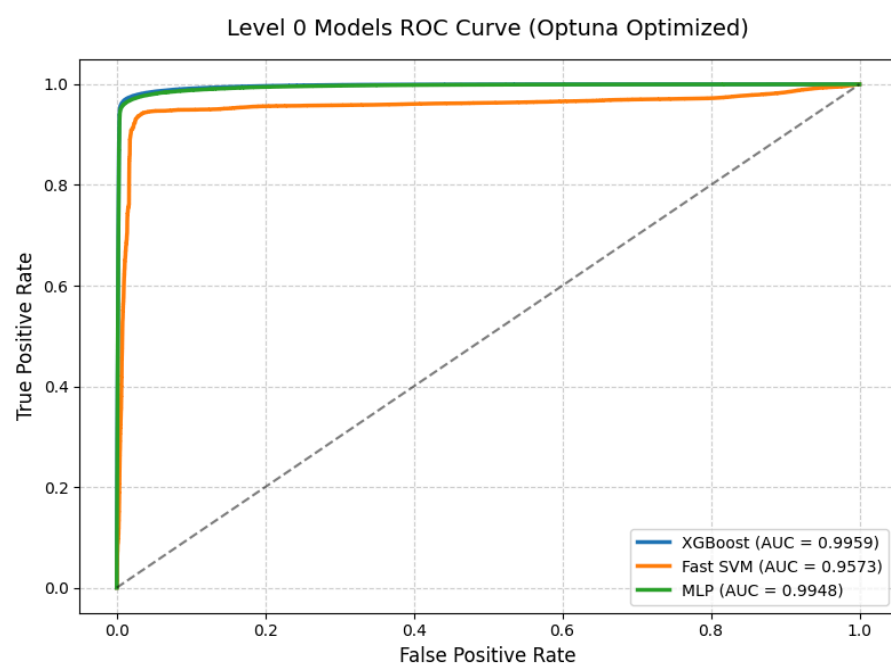
本節定義系統核心 AI 模型之訓練流程，包含異質基模型之訓練策略、自動化超參數調優以及最終元模型之決策融合實作。

4.7.1 基礎層模型實驗與最終選型

為了決定基礎層(Level 0)的最佳演算法，本專案針對URL、HTML與AI語意三路數據分別執行了演算法對比實驗。

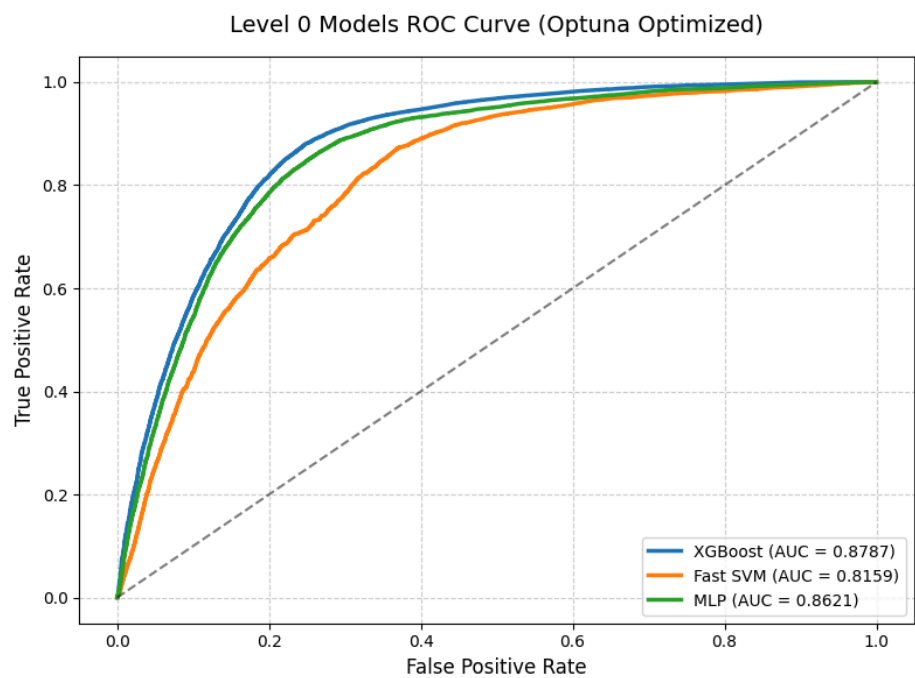
- **對比演算法：**針對每一模態均獨立測試 XGBoost、Fast SVM (SGDClassifier) 以及 MLP (Multi-layer Perceptron)，詳細演算法參考系統計畫書附錄10.5。
- **實驗結論：**
 - URL(192611筆資料)：

模型	最佳參數	最佳結果(ROC AUC)
XGBoost	{'n_estimators': 500, 'max_depth': 10, 'learning_rate': 0.029575563409918144, 'subsample': 0.8516126313082992, 'colsample_bytree': 0.6094457501991484}	0.9959
Fast SVM	{'alpha': 1.0225500505405769e-06}	0.9573
MLP	n_layers: 2, n_units_l0: 160, n_units_l1: 96 learning_rate_init: 0.0010814230485219653 alpha: 0.0002201499120134984	0.9947



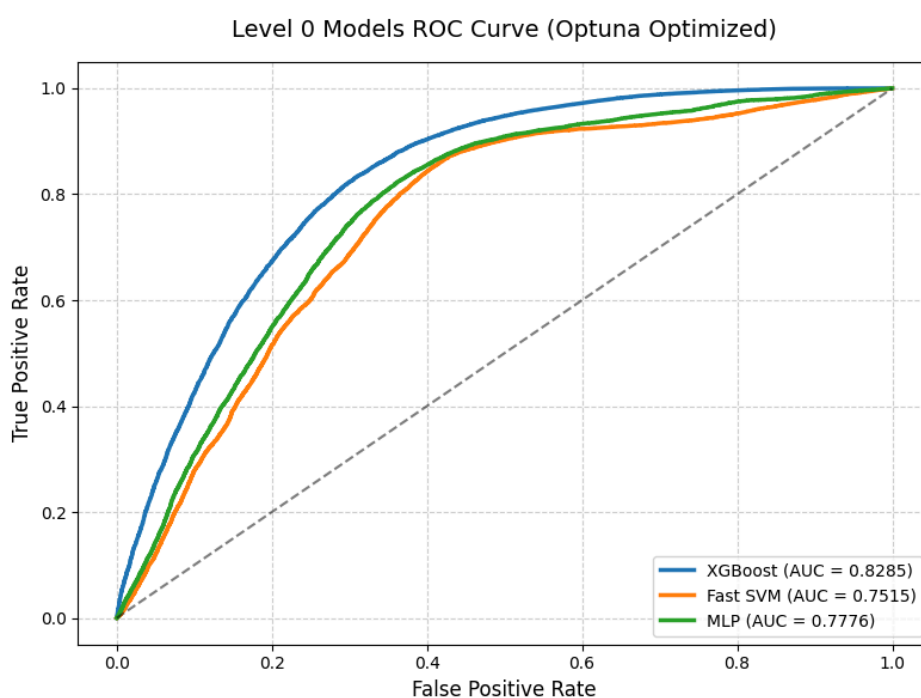
○ HTML(71005筆資料):

模型	最佳參數	最佳結果(ROC AUC)
XGBoost	{'n_estimators': 400, 'max_depth': 9, 'learning_rate': 0.017971680680970523, 'subsample': 0.9878504930154548, 'colsample_bytree': 0.75892709611766}	0.8785
Fast SVM	{'alpha': 8.635953655972402e-06}	0.8159
MLP	n_layers: 2, n_units_10: 192, n_units_11: 192 learning_rate_init: 0.00411325762702593 alpha: 0.0002201499120134984	0.8621



- AI(81036筆資料):

模型	最佳參數	最佳結果(ROC AUC)
XGBoost	{ 'n_estimators': 500, 'max_depth': 9, 'learning_rate': 0.01104464833671941, 'subsample': 0.6052893940100675, 'colsample_bytree': 0.7235210399820414 }	0.8285
Fast SVM	{ 'alpha': 0.007594939952314412 }	0.7515
MLP	n_layers: 1 n_units_l0: 192 learning_rate_init: 0.00023962099362217815	0.7776



- 經由Optuna進行超參數最佳化後，XGBoost在三路模態的ROC AUC與F1-Score指標上均顯著優於其他模型。
- **選型決策：**基於實驗結果，系統之 Level 0三路預測路徑統一採用XGBoost 作為核心預測引擎，以確保在高維特徵下的推論精準度。

```
print("正在訓練最終 XGBoost...")
final_models["XGBoost"].fit(X, y)
final_models["XGBoost"].save_model("optuna_xgb_ai_final.json")
print("  ✅ 已儲存: optuna_xgb_ai_final.json")
```

4.7.2 自動化超參數調優規範

系統利用 Optuna 對所有模態的模型進行優化。

- XGBoost 關鍵搜尋參數：

```
"n_estimators": trial.suggest_int("n_estimators", 100, 600, step=100),
"max_depth": trial.suggest_int("max_depth", 3, 15),
"learning_rate": trial.suggest_float("learning_rate", 0.01, 0.2, log=True),
"subsample": trial.suggest_float("subsample", 0.6, 1.0),
"colsample_bytree": trial.suggest_float("colsample_bytree", 0.6, 1.0),

# 固定參數
"objective": "binary:logistic",
"eval_metric": "auc",
"tree_method": "hist",
"scale_pos_weight": ratio,
"random_state": 42
```

- 優化目標：以5-Fold分層交叉驗證（Stratified K-Fold）之ROC AUC平均值作為最佳化指標。

4.7.3 元模型訓練與權重整合

- Stacking 整合邏輯：

- 收集三路XGBoost模型產出之預測機率 $P\{\text{url}\}$, $P\{\text{html}\}$, $P\{\text{ai}\}$ 。

```
y_pred_proba_url = xgb_url_model.predict_proba(X_url_test)[: , 1]
y_pred_proba_html = xgb_html_model.predict_proba(X_html_test)[: , 1]
y_pred_proba_ai = xgb_ai_model.predict_proba(X_ai_test)[: , 1]
```

- 結合機率特徵，餵給 Logistic Regression 元模型學習權重。

```
X_meta = np.c_[y_pred_proba_url, y_pred_proba_html, y_pred_proba_ai]
y_meta = y

meta_model = LogisticRegression(class_weight='balanced')
meta_model.fit(X_train_meta, y_train_meta)
```

- 模型持久化：

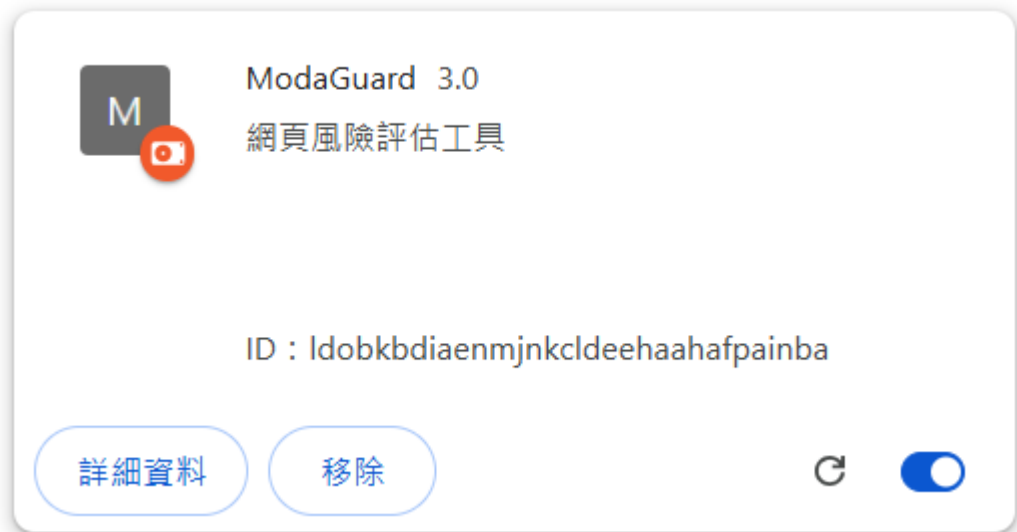
- 訓練完成後，各路L0模型儲存為url_xgb_v1.json、html_xgb_v1.json、ai_xgb_v1.json。
- 元模型儲存為meta_model_logistic_v1.pkl。

4.7.4 實作邏輯演算法

此部分參考繼統計畫書附錄之10.5。

五、系統使用流程(System Usage Process)

1. 新增擴充功能



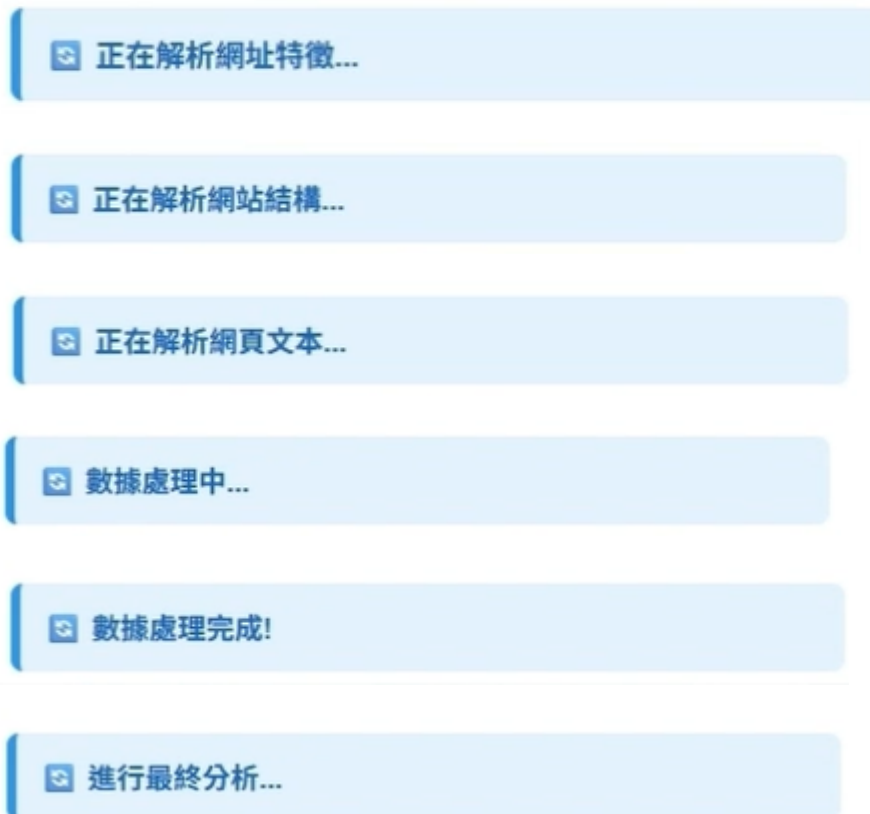
2. 點擊使用此功能



3. 選擇一鍵掃描



4. 等待系統執行



5. 檢視評估報告

網站相對安全

釣魚風險指數: **0.51%**

AI評估：

該網站目前評估為相對安全。雖存在網頁標題不符、原始碼含常見釣魚關鍵字及缺乏聯絡資訊等疑點，但整體風險評估值僅0.51%，未達高風險門檻。建議持續觀察網站更新與安全認證狀態，現階段可安心瀏覽。

參、測試計畫(Testing)

本章節詳述本專案「ModaGuard」偵測系統的測試流程與實驗結果。透過量化的模型效能指標與質化的實務診斷報告，驗證多模態堆疊架構在釣魚偵測上的有效性與系統穩定性。

內容目錄

- 一、測試目的
- 二、測試環境
- 三、單一模型效能測試實驗
- 四、堆疊模型效能驗證實驗
- 五、實務情境診斷測試
- 六、邊緣端效能測試
- 七、測試結論

一、測試目的

本測試計畫主要目的有三：

1. **單一模型效能測試：**確保基模型的模型選擇與參數配置是否以最佳化。
2. **堆疊模型效能驗證：**揚棄傳統單一指標的評估方式，透過嚴格切分之獨立測試集，驗證系統所採用之多模態堆疊架構之優勢。除了檢視常規之準確率、精準率與召回率外，更進一步導入K-S統計量與機率分化指標，具體驗證堆疊機制如何有效整合各特徵（URL、HTML、網頁文本）之權重，消除單一模型之決策猶豫區間，從而提供具備極高自信度與穩定性之防禦預測。
3. **實務情境診斷測試：**考量系統面對終端使用者之實用性，驗證系統內建之SLM所自動產出之「網站風險評估報告」。透過檢驗其生成內容之邏輯一致性與語意準確度，確保系統不僅能輸出冷硬的風險機率，更能提供具備高度「可解釋性」之具體威脅特徵描述（如：明確指出釣魚特徵或安全佐證），以建立終端使用者對系統之信任感。
4. **邊緣端效能測試：**由於軟硬體環境限制嚴苛，本系統不能以非常高效之速度執行，也因此在本次開發著重探討此部分並沒有重大意義，但仍列出此次系統開發之對於邊緣端運算效能測試之大致數據，以利後續改良。

二、測試環境

- **硬體設備：**
 - 處理器：11th Gen Intel(R) Core(TM) i7-11800H。
 - 記憶體：16.0 GB。
 - 顯示卡：NVIDIA GeForce RTX 3060 Laptop GPU
[註：依系統運行 SLM 之需求配置]。
 - 系統類型：64 位元作業系統，x64 型處理器。
- **軟體環境：**
 - 作業系統：Windows 11 Professional (v23H2)。
 - 運算驅動：CUDA 12.1。
 - 開發語言：Python 3.13。
 - 瀏覽器環境：Google Chrome v120.0+ (支援 Manifest V3)。
 - 核心框架：Flask 3.0, llama-cpp-python, XGBoost, TensorFlow 2.10。
- **測試集規格：**使用系統規格書中4.6.3節中Meta_Set切出的20%做為測試集，確保模型未曾接觸過測試數據。
- **異質資料集：**本研究特意整合了兩份不同來源、不同標註標準的數據集（Kaggle 與 NIST ），旨在測試模型在異質資料集下的泛化能力。

三、單一模型效能測試實驗

旨在驗證「網址、結構、語意」三路特徵模型效能與銓重調配是否是最佳。

3.1 實驗設計

使用Optuna模組，讓其在一定區間內尋找最佳參數，並同時比較XGBoost、Fast SVM、MLP三個模型各自的最佳結果(使用ROC AUC作為指標)。

3.2 實驗方法

1. 導入模組與訓練測試集。
2. 設定需標準化或需另外前處理的欄位。
3. 設定預探索的參數區間。
4. 使用StratifiedKFold進行訓練與預測。
5. 取得各模型之結果，並保留最好的作為實務上使用之基模型。

註:具體演算法參考「系統計畫書之附錄10.5」

3.3 實驗結果(ROC AUC)與結論

3.3.1 實驗結果

1. URL (192611筆資料) :
 - a. XGBoost: 0.9959
 - b. Fast SVM: 0.9573
 - c. MLP: 0.9947
2. HTML (71005筆資料) :
 - a. XGBoost: 0.8785
 - b. Fast SVM: 0.8159
 - c. MLP: 0.8671
3. AI Text (81036筆資料):
 - a. XGBoost: 0.8285
 - b. Fast SVM: 0.7515
 - c. MLP: 0.7776

3.3.2 結論

根據此結果，三個模型皆使用XGBoost作為基模型。

註:

相關數據(如權重配置)參考「系統規格書」之第4.7節「機器學習模型訓練與建置」。

四、堆疊模型效能驗證實驗

旨在驗證整合「網址、結構、語意」三路特徵後，對於提升偵測率的具體貢獻。

4.1 實驗設計

分別訓練四個模型進行對比，並透過指標與圖表觀察對同一測試集表現結果(10918筆資料)。

1. URL Model：僅使用 45 維網址統計特徵。
2. HTML Model：僅使用 16 維 DOM 結構特徵。
3. AI Text Model：僅使用 15 維 SLM 語意特徵。
4. Meta(Stacking) Model：採用Stacking架構整合上述三路特徵。

4.2 實驗方法

1. 模型與資料載入

```
import joblib
from xgboost import XGBClassifier
xgb_url_model = XGBClassifier()
xgb_url_model.load_model('/content/optuna_xgb_url_final.json')
xgb_html_model = XGBClassifier()
xgb_html_model.load_model('/content/optuna_xgb_html_final.json')
xgb_ai_model = XGBClassifier()
xgb_ai_model.load_model('/content/optuna_xgb_ai_final.json')
meta_model = joblib.load('/content/meta_logistic_final.pkl')
```

```
df = pd.read_csv('/content/meta_test.csv')
```

2. 將三路特徵與標籤進行分類

```
url = df_con[['length_url', 'length_hostname', 'ip', 'nb_dots',
             'nb_hyphens', 'nb_at', 'nb_qm', 'nb_and', 'nb_or', 'nb_eq',
             'nb_underscore', 'nb_tilde', 'nb_percent', 'nb_slash', 'nb_star',
             'nb_colon', 'nb_comma', 'nb_semicolumn', 'nb_dollar', 'nb_space',
             'nb_www', 'nb_com', 'nb_dslash', 'http_in_path', 'https_token',
             'ratio_digits_url', 'ratio_digits_host', 'punycode', 'port',
             'tld_in_path', 'tld_in_subdomain', 'nb_subdomains',
             'abnormal_subdomain', 'prefix_suffix', 'path_extension',
             'length_words_raw', 'char_repeat', 'shortest_word_host',
             'shortest_word_path', 'longest_words_raw', 'longest_word_host',
             'longest_word_path', 'avg_words_raw', 'avg_word_host', 'avg_word_path'
            ]]
html = df_con[['phish_hints', 'domain_in_brand', 'nb_hyperlinks', 'ratio_intHyperlinks',
               'ratio_extHyperlinks', 'ratio_extRedirection', 'ratio_extErrors',
               'external_favicon', 'links_in_tags', 'ratio_extMedia', 'safe_anchor',
               'empty_title', 'domain_in_title', 'domain_with_copyright',
               'has_meta_refresh', 'has_js_redirect'
              ]]
ai = df_con[['creates_urgency', 'uses_threats', 'requests_sensitive_info',
             'offers_unrealistic_rewards', 'has_spelling_grammar_errors',
             'impersonated_brand', 'has_valid_copyright_year',
             'is_content_login_focused', 'has_rich_navigation',
             'has_physical_address', 'has_phone_number',
             'content_consistency_score', 'language_professionalism_score',
             'overall_phishing_likelihood_score', 'text_length'
            ]]
y_true = df_con['target']
```

3. 對URL、HTML和AI Text Model進行預測，並觀察分布

```
y_pred_proba_url = xgb_url_model.predict_proba(url)[: , 1]
y_pred_proba_html = xgb_html_model.predict_proba(html)[: , 1]
y_pred_proba_ai = xgb_ai_model.predict_proba(ai)[: , 1]

# 1. 繪製 URL 模型的分布
plot_model_distribution(y_true, y_pred_proba_url, "URL Features")

# 2. 繪製 HTML 模型的分布
plot_model_distribution(y_true, y_pred_proba_html, "HTML Features")

# 3. 繪製 AI 文本特徵模型的分布
plot_model_distribution(y_true, y_pred_proba_ai, "AI Text Features")
```

4. 將上述預測結果拿來預測Meta Model，並觀察分布

```
meta = np.c_[y_pred_proba_url, y_pred_proba_html, y_pred_proba_ai]
y_pred_proba_meta = meta_model.predict_proba(meta)[: , 1]
plot_model_distribution(y_true, y_pred_proba_meta, "Stacking Features")
```

5. 指標比較

```

# 1. 計算 ROC AUC (這是不受 threshold 影響的指標，只看模型排序能力)
auc = roc_auc_score(y_true, y_pred_proba)

# 2. 根據門檻 (threshold) 將機率轉換成 0 或 1 的硬標籤
y_pred_label = (y_pred_proba >= threshold).astype(int)

# 3. 計算各項分類指標
acc = accuracy_score(y_true, y_pred_label)
prec = precision_score(y_true, y_pred_label, zero_division=0)
rec = recall_score(y_true, y_pred_label, zero_division=0)
f1 = f1_score(y_true, y_pred_label, zero_division=0)

# 1. 計算 K-S Statistic (分化距離，數值越大越好)
# 將機率分為 Class 0 和 Class 1
proba_class_0 = y_pred_proba[y_true == 0]
proba_class_1 = y_pred_proba[y_true == 1]

# 使用 scipy 計算 K-S 統計量
ks_stat, _ = ks_2samp(proba_class_0, proba_class_1)
ks_score = ks_stat * 100 # 轉換為百分比 (0~100)

# 2. 計算猶豫區間比例 (落在 0.4 ~ 0.6 的資料比例，數值越低越好)
ambiguous_count = np.sum((y_pred_proba >= 0.4) & (y_pred_proba <= 0.6))
ambiguity_ratio = (ambiguous_count / len(y_pred_proba)) * 100

# 3. 計算 Brier Score (機率均方誤差，數值越低越好)
brier = brier_score_loss(y_true, y_pred_proba)

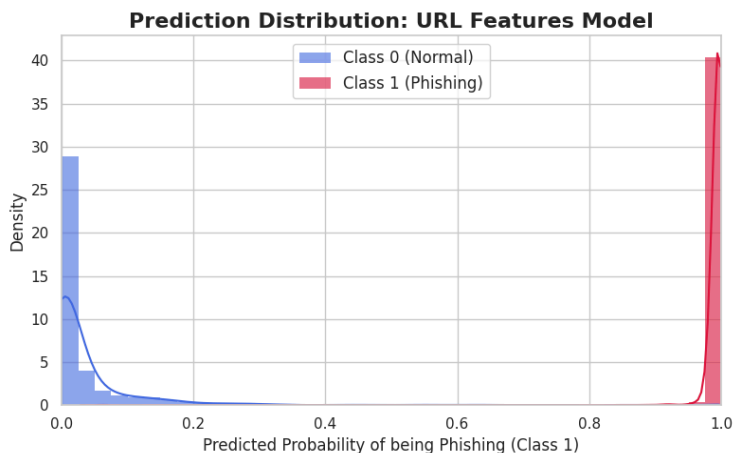
# 4. 計算高自信區間 (大於0.9 或 小於0.1 的比例，數值越高越好)
confident_count = np.sum((y_pred_proba >= 0.9) | (y_pred_proba <= 0.1))
confidence_ratio = (confident_count / len(y_pred_proba)) * 100

```

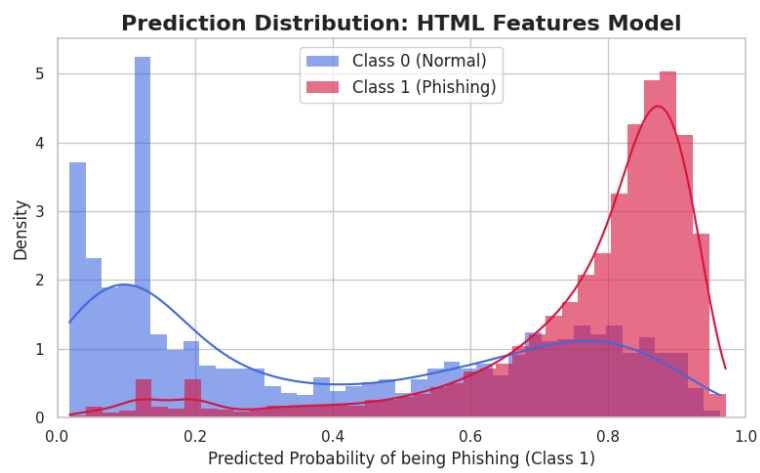
4.3 實驗結果與效能數據

4.3.1 各模型分布狀況

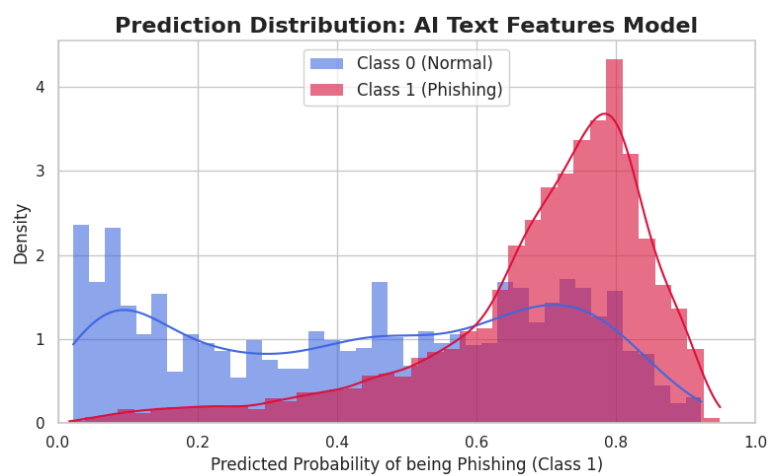
1. URL model



2. HTML Model



3. AI Text Model



4.3.2 指標比較

	Model	ROC AUC	Accuracy	Precision	Recall	F1-Score
0	URL Model	0.9981	0.9947	0.9985	0.9954	0.9970
1	HTML Model	0.8384	0.8710	0.9427	0.9088	0.9255
2	AI Text Model	0.7747	0.8333	0.9339	0.8726	0.9022
3	Meta Model	0.9987	0.9945	0.9989	0.9949	0.9969

(表一)

	Model	K-S Score (↑)	Brier Score (↓)	Ambiguity Ratio (%) (↓)	High Confidence Ratio (%) (↑)
0	URL Model	99.00	0.0043	0.17%	98.13%
1	HTML Model	53.09	0.1100	6.92%	18.39%
2	AI Text Model	43.09	0.1415	14.63%	4.77%
3	Meta (Stacked) Model	98.98	0.0050	0.06%	99.69%

(表二)

4.4 結果討論

4.4.1 常規效能評估結果討論(4.3.2之表一)

從ROC AUC中，能夠觀察到URL模型展現了極高的防禦能力，證明網址結構對於釣魚網站辨識是極為有效的判斷因素。相對而言，HTML模型與AI文本模型在單獨運作時則表現較為薄弱，評估是受制於特徵擷取的完整性與雜訊的干擾(如:彈跳視窗、機器人驗證與其他反爬蟲機制等)。

然而，當系統導入堆疊模型進行多模態特徵融合後，雖然整體Accuracy與Recall呈現微幅下滑(幅度小於0.05%)，但在資安實務中，ROC AUC的提升更具指標意義。這代表模型在面對非均衡樣本或對抗性攻擊時，具備更穩健的閾值選擇彈性，有效平衡了攔截率與誤報率。

4.4.2 機率分佈可視化與決策極化現象探討(4.3.1之分布圖與4.3.2之表二)

為深入驗證堆疊模型在決策過程中的穩定性與確定性，本專案繪製了各模型的預測機率分布，並搭配進階機率分布指標(表二)進行量化分析。本次實驗共有以下之發現：

1. 單一模型的猶豫與重疊區間：

觀察HTML與AI文本模型之機率分布，可明顯發現正常(Class 0)與釣魚(Class 1)的雙峰較URL呈現扁平且高度重疊。根據表二數據顯示，這二個模型之猶豫區間比例(Ambiguity Ratio)高達6.92%與14.63%，且高自信區間比例(High Confidence Ratio)極低，難以做為系統自動攔截的絕對依據。

2. 堆疊模型的極化效應與自信：

從Stacking Feature Model的機率分布即可發現兩類別之波峰極端地推向兩側，中間過度帶幾乎消失。但若單論雙峰分化現象無法展示出堆疊模型取代單一URL模型所能提供的額外效益。進一步比對表二，即可發現堆疊模型展現了更為極端的決策極化效應：

- 堆疊模型將猶豫區間比例進一步壓縮至僅剩0.06%
- 堆疊模型的高自信區間比例成長至99.69%

此數據證明了異質模型間具備特徵互補性，也代表堆疊模型在系統做出預測時，具備了絕對的決策把握。

4.4.3 指標微幅下降之學理推斷

堆疊模型的K-S Score與 Brier Score較單一URL模型出現了微幅的退化。此現象推斷應不為效能低落的緣故，而是機率校準與極化之權衡：

- **Brier Score 微升原因：**由於模型被推向高自信，當對於少數誤判案例時計算均方誤差的Brier Score會給予更重的懲罰。
- **K-S Score 微降原因：**K-S統計量衡量的是完美排序能力。堆疊模型融合HTML與AI這二個具有較高模型訊號的輔助特徵後，必然會對於排序產生擾動。

4.4.4 綜合結論與縱深防禦價值

儘管URL模型在部分指標有著微幅優勢，然本系統仍然採用堆疊模型架構，其核心是基於以下三點：

- **量化效益：**

綜觀上述指標，單一URL模型在K-S分數上僅以0.02的微幅差距領先。然而堆疊模型在高自信區間的成長高達1.56%，並將猶豫區間降至趨近於零。相較之下，堆疊模型所帶來的效益遠大於少數指標的微小折損。對於需要明確決策的防禦系統而言，是極具實用價值的存在。

- **精準率的提升：**

堆疊模型犧牲較小的排序完美度，換來更高的精準度。對於本專案而言，降低假陽性率避免誤報是維持使用者體驗的重要原則。堆疊模型藉由HTML、AI特徵作為輔助核對機制，能成功校正被URL單一特徵誤判的邊緣網站。

- **建構縱身防禦：**

在真實資安攻防中，單一URL模型極易受到針對性逃逸攻擊(如:合法短網址、高信譽網站被駭入等等)。堆疊模型不依賴單一特徵，當攻擊者嘗試繞過網址特徵檢查，系統仍能藉由網頁結構與語意文本進行判斷。

總結而言，本系統之堆疊架構能夠有效吸收多模態特徵，以較小的校準度作為代價，換取更強大的抗規避能力、極低的決策猶豫率，以及更符合實務需求的防護穩定性。本實驗證明堆疊架構取代單一模型投入實務應用的高度可行性。

五、實務情境診斷測試

本節旨在驗證 ModaGuard 在面對真實網頁樣本時，其風險指標與 AI 評估報告的邏輯一致性。

5.1 測試方法與安全規範

考慮到釣魚網站隱含的危險性(病毒、攻擊等)，本測試方法根據標籤有所不同。

1. 正常網站：直接透過ModaGuard插件於實體瀏覽器中執行採集與分析。
2. 釣魚網站：為確保測試環境安全性，本研究直接從後端資料庫調用以確認標籤的惡意原始數據，並透過API模擬注入到偵測系統。

5.2 診斷案例分析

案例類型	風險指數	AI診斷理由
正常網站A	0.51%	AI評估： 該網站目前評估為相對安全。雖存在網頁標題不一致、原始碼含常見釣魚關鍵字及缺乏聯絡資訊等疑點，但綜合分析顯示其風險屬低機率事件。系統未偵測到惡意程式或主動攻擊行為，且無證據顯示網站已遭入侵或用於詐欺活動，可放心瀏覽。
正常網站B	4.62%	AI評估： 該網站目前未發現顯著安全風險。雖存在少量可疑特徵如自動跳轉指令與常見釣魚關鍵字，但均未觸發實際危害行為。網頁結構完整且無惡意載入行為，標題與網域名稱具關聯性，聯絡資訊缺失可能為小型網站常見現象。綜合評估顯示網站安全性符合一般標準，可放心瀏覽。
釣魚網站A	99.46%	原因： 該網站具有極高釣魚風險！網頁標題空白且與網域名稱無關聯，缺乏品牌一致性，同時未提供實體地址與聯絡電話等必要資訊，顯示來源可疑。請立即停止訪問並切勿輸入任何個人資料，以免遭受詐騙或資料洩漏。
釣魚網站B	99.50%	reason: 該網站具有極高風險！網頁標題空白、原始碼藏匿login/verify等釣魚關鍵字，且無實體地址與聯絡資訊，顯示為高風險釣魚網站。請立即離開並切勿輸入任何個人資料，避免遭竊取敏感資訊。

5.3 測試結論

實驗顯示，系統能根據不同網站特徵給出區別性的評估。使用者能透過AI評估理解該系統的大致判斷理由，證明了ModaGuard在可解釋性資安防禦上的實用性。

六、邊緣端效能測試

6.1 推論時延分析

本系統之推論時間影響因素包含:前後端訊息傳遞、資料處理、模型預測與AI評估。其中，資料處理中的網頁文本特徵解析與AI評估佔據了大部分的時間(約八成)。網頁文本特徵解析時間又會根據文本長度有所不同。在整個系統實作過程，文本越長所花費的AI解析時間越多，具體而言約落在25至65秒左右。而在AI評估階段，則會因為理由清單之長短影響傳入的文本長度，但差距相較分析階段較小，時間約落在25至35秒間。總體而言，從使用者執行掃描到產出結果，時間約落在60~110秒左右。

6.2 資源占用分析

本系統採用了經 4-bit 量化優化之 GGUF 格式模型，以在消費級顯卡（RTX 3060）上達成記憶體佔用與推論速度的平衡。為加快推論速度，使用GPU進行運算，然因此模型原本大小與 Token 輸入量的關係，再加上設備並不是很先進(RTX 3060)，時常導致GPU瞬間占滿而降速至CPU。相較之下，若處理過程不經過語言模型推論，時間與資源消耗情形較輕微。硬體負載與推論速度所給予的使用者體驗將成為未來此系統之核心解決目標之一。雖然目前推論時延較長，但本研究重點在於驗證地端 SLM 於資安診斷之可行性。未來可透過模型剪枝或更高效之推論引擎進一步優化。

七、測試結論

ModaGuard系統在整合網址、網頁結構極網頁文本三模態之特徵後，成功提高了釣魚網站辨識之堆疊模型其實務上之價值，不論是在判斷，抑或是在對於判斷產出的自信與猶豫上，皆有所改善。透過實驗，證明本系統不僅具備優異的量化指標，更能提供邏輯解釋力的AI診斷報告，在保護使用者隱私與安全的前提下，達成了技術與實務兼具的研發目標。

肆、使用者文件說明(User Document)

本手冊旨在引導使用者安裝、配置並操作「ModaGuard」多模態釣魚偵測系統。本系統採用本地化邊緣運算技術，確保所有網頁分析流程皆在您的裝置上執行。

內容目錄

- 一、系統環境要求
- 二、系統安裝與配置步驟
- 三、使用流程與介面操作
- 四、常見問題
- 五、故障排除

一、系統環境要求

在安裝本系統前，請先確保硬體與軟體規範是否符合(此部分僅做為參考，具體運行成效請依實務為準)：

- 作業系統：Windows 10/11。
- 硬體要求：
 - GPU：NVIDIA GeForce RTX 30系列或更高（建議具備8GB以上VRAM）。
 - RAM：16GB 或以上。
- 軟體依賴：
 - Google Chrome：建議更新至 v120 以上版本。
 - NVIDIA 驅動程式：須支援 CUDA 12.1。

二、系統安裝與配置步驟

2.1 啟動本地推論後端

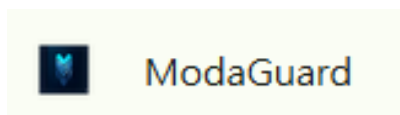
1. 解壓縮「phishing-backend」資料夾。
2. 確保已安裝相關須執行之模組(Python、llama_cpp、flask等)。
3. 執行python [app.py](#)
4. 終端介面顯示「active」即代表成功啟動

2.2 安裝Chrome擴充功能

1. 開啟Chrome瀏覽器，進入管理擴充功能主頁。
2. 開啟右上角「開發人員模式」。
3. 點擊「載入未封裝項目」，選擇「phishing-detector-extension」資料夾。



4. 即可在工具找到「ModaGuard」並使用。



三、使用流程與介面操作

3.1 一鍵安全掃描

當需要此工具時在工具列點擊ModaGuard圖示，系統即跳出視窗，點選「一鍵安全掃描」按鈕即開始掃描。

3.2 即時進度追蹤

系統透過NDJSON技術時時顯示當前的執行階段，包含：



3.3 解讀偵測報告

掃描完成後，介面顯示最終結果(風險機率和AI評估理由)：



四、常見問題

Q1：為甚麼掃描時間很長？

Answer：ModaGuard的運行仰賴本地資源，若軟硬體規格不夠將導致推論時間增加。

Q2：可以使用其他語言模型嗎？

Answer：可以，只是須確保模型所在的路徑與程式內部有所匹配，且硬體足以負擔此模型運算。

Q3：ModaGuard會蒐集所瀏覽的網頁資料嗎？

Answer：不會，本系統不會蒐集使用者之數據，使用者數據僅在掃描階段使用，用完即棄。

Q4：一定要自主安裝Chrome插件嗎？

Answer：ModaGuard當前尚未有著實體伺服器，使用者端需自行布置所有運行所需要的資源與環境。

五、故障排除

根據開發階段所經歷的一些問題統整出可能的故障原因。

錯誤現象	可能原因	排除方法
安裝llama-cp時出現錯誤	尚未安裝驅動程式或是版本不符。	檢查驅動是否安裝，並切換對應版本。
運行時出現錯誤 (NoneType)	為正確抓取到網頁資訊。	當前網頁可能有著超出後端系統可容納的數據量上限，，若設備允許，可以提高語言模型的Token量(n_ctx)。
啟動後電腦變很卡	當前硬體對於此特規格的系統負載能力不足。	調降系統規格，尤其是語言模型，抑或是減少同時間的其他應用程序執行。

伍、結論與未來展望 (Conclusion & Future Work)

本專案「ModaGuard」透過多模態特徵整合與邊緣運算技術，成功構建了一個兼具高準確度與隱私保護的釣魚偵測系統。本章節將總結專案成果、反思當前限制，並提出未來的改良方向與研究參考。

內容目錄

- 一、專案總結
- 二、系統限制與不足
- 三、未來展望
- 四、參考資料

一、專案總結

本研究成功實作了基於 Stacking 集成架構的偵測引擎，並達成以下核心目標：

1. **多模態特徵融合與決策優化：**整合了網址、DOM結構與sLM文本語意共76維特徵，經實驗證明最終集成模型效能優於單一模型，將「猶豫區間」壓縮至0.06%，顯著提升自動化決策穩定度，驗證多維度分析在資安防禦上的必要性。
2. **可解釋型AI：**將語言模型與偵測引擎結合，將傳統機器學習的「黑盒」機率轉化為具備資安邏輯的繁體中文診斷報告。透過知識驅動(專家規則)與數據驅動(AI 模型)的解構整合，提升了使用者對系統的信任感。
3. **本地化邊緣防護：**利用CUDA加速與 GGUF 量化模型部屬實現全本地推論，在「數據不離地」的原則下，解決雲端 LLM 高昂的 API 成本與個資外洩風險。

二、系統限制與不足

儘管本系統表現優異，但在實務開發與測試中仍發現以下瓶頸：

1. **硬體資源門檻：**由於本地運行之 SLM 需佔用約 5.8GB 的 VRAM，對於缺乏獨立顯卡的文書型筆電用戶或是型號較舊之處理器而言，進入門檻較高。
2. **推論延遲：**系統整體運作時長約60-110秒，主要瓶頸在於語言模型推論時間，對於即時掃描的訴求仍有待優化。
3. **長文本處理壓力：**當網頁內容較龐大時，文字特徵萃取的耗時會增加，且可能超出SLM的上下文窗口限制。
4. **樣本時效性與概念飄移：**目前訓練樣本仰賴公開資料及提供之網址，面對快速演進的動態釣魚技術，模型可能存在特徵退化的風險。
5. **模型效能差異：**HTML及AI文本之基模型其效能相較URL基模型稍顯不足，顯示若特徵若處理不當，可能會對整體排序能力產生微幅擾動。
6. **可解釋性的程度：**儘管能夠針對風險機率與網站特定特徵進行適性化的評估，但其特定特徵之判定(也就是理由清單)，可能不夠細膩與多元。

三、未來展望

針對上述不足，本專案計畫於後續版本進行以下改良：

1. **模型蒸餾與極致量化：** 研究將 Qwen-8B 蒸餾為專門針對「網頁意圖分析」的小型專用模型，將推論時間壓縮，抑或是選擇其他開源模型作為替代。
2. **混合式動態分析架構：** 實作「預先過濾與深度診斷」的兩階段機制，對於 URL 掃描即可判定的明顯威脅立即產出報告；對於疑似社交工程之複雜網頁，才使用 SLM 進行深度語意分析，以平衡效能(語言模型推論耗時之問題)與防護深度。
3. **對抗性攻擊防禦強化：** 研究針對「提示詞注入」的攔截器，防止攻擊者透過網頁內容誤導 SLM 的判斷邏輯。
4. **跨平台生態系建立：** 將 ModaGuard 核心移植至行動端瀏覽器，並探索與其他系統結合的可能性，全方位攔截釣魚威脅。
5. **安全沙盒預瀏覽系統：** 結合 Docker 技術實現安全隔離，提供使用者在進入可疑網頁前的「虛擬代瀏覽」回饋，達成真正的零風險預防。
6. **多維特徵演進：** 擴大 HTML 與網頁文本相關特徵維度，提升模型整體辨識能力。
7. **自動化樣本更新機制：** 研發自動化爬蟲對接即時威脅情報流，透過線上學習或定期重新訓練，對抗資安領域的概念漂移問題。

陸、參考資料 (References)

1. 資料集與樣本
 - a. [Prasad, A., & Chandra, S. \(2024\). PhiUSIIL Phishing URL. UCI Machine Learning Repository](#)
 - b. [Shashwat Tiwari. \(2021\) Web page Phishing Detection Dataset. Kaggle](#)
2. 開源模型(Qwen3-8B-Q4_K_S)
 - a. <https://huggingface.co/unsloth/Qwen3-8B-GGUF>
3. 參考文獻
 - a. [A Review of Phishing Detection Research\(XIE Lixia1 , ZHANG Hao2 , YANG Hongyu2* , HU Ze2 , CHENG Xiang3, 4, and ZHANG Liang\)](#)
 - b. [Web page Phishing Detection\(黃彥蓉\)](#)
 - c. [Anti-Phishing Working Group \(APWG\)](#)
 - d. [Phishing for User Context: Understanding the NIST Phish Scale\(Shanee Dawkins, Jody Jacobs\)](#)
 - e. [淺談釣魚攻擊之釣魚網站\(林子南\)](#)
 - f. [網路釣魚特徵\(ILTM 研究團隊\)](#)